

信号与系统

第十九讲 深度卷积神经网络

谷源涛

清华大学电子工程系

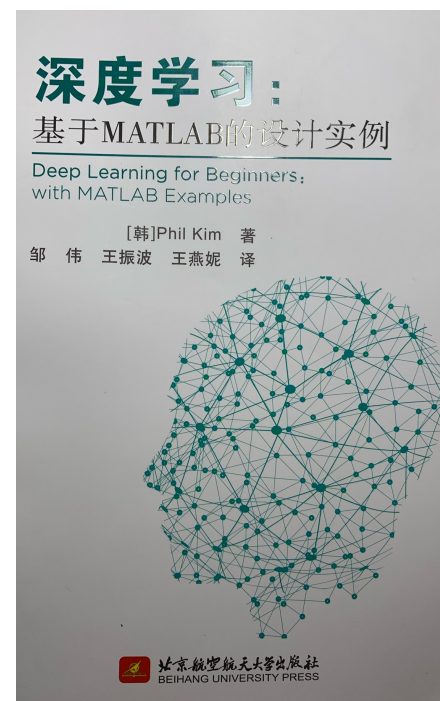
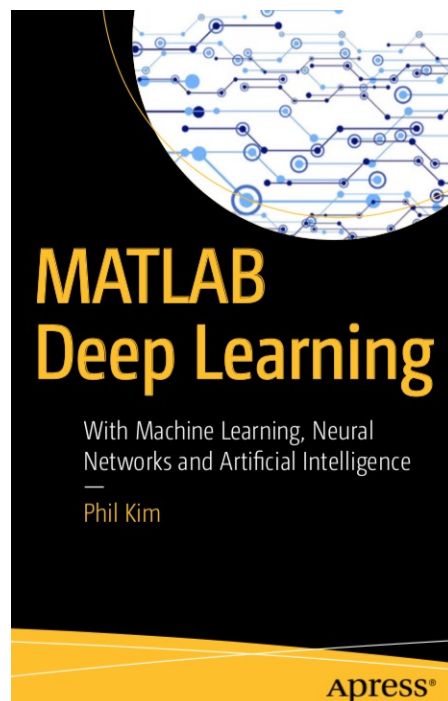
2024 年 春季学期

主要内容

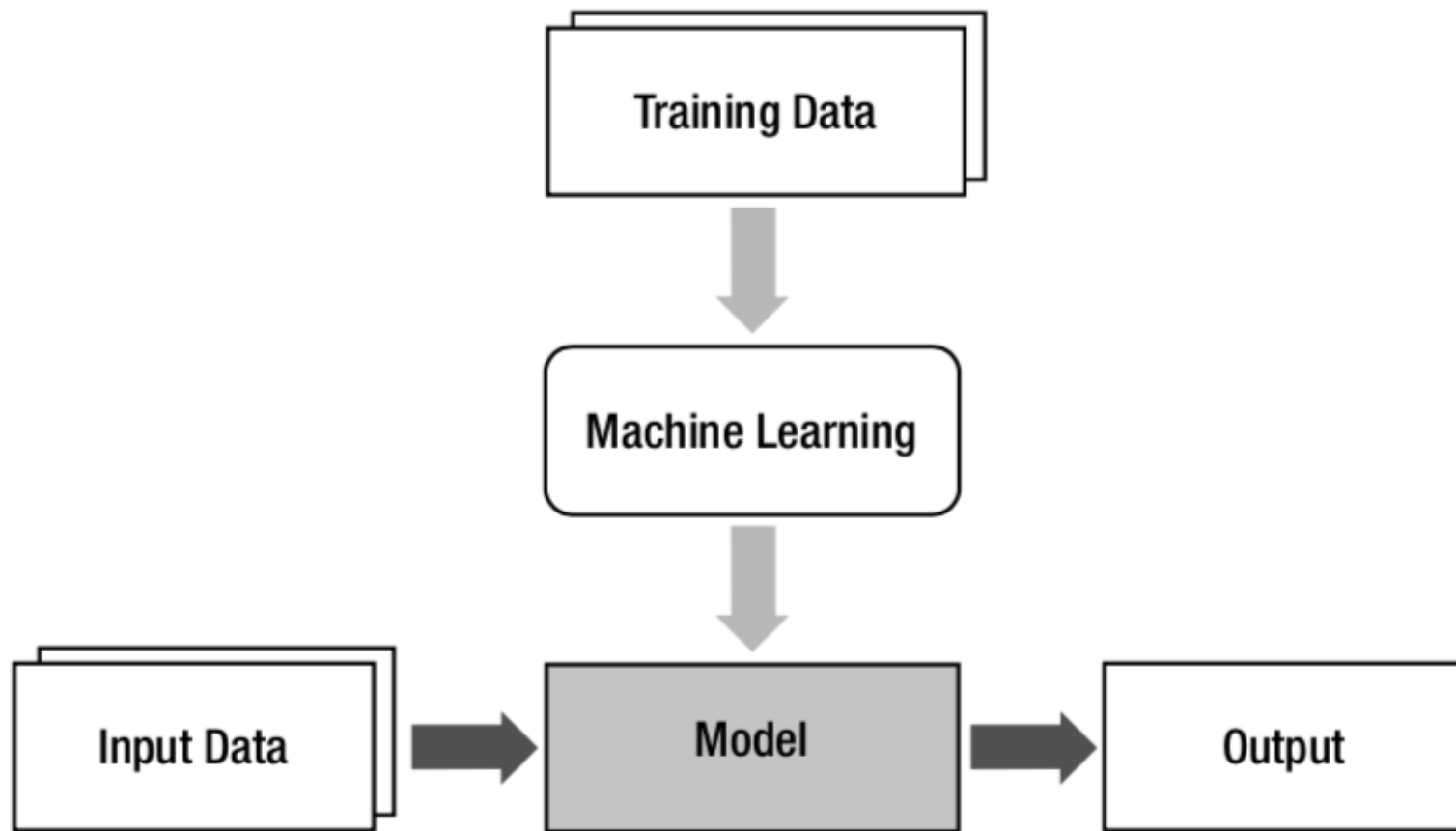
- 机器学习
- 神经网络
- 训练多层神经网络
- 用于分类的多层神经网络
- 深度学习
- 卷积神经网络

主要内容

- 机器学习
- 神经网络
- 训练多层神经网络
- 用于分类的多层神经网络
- 深度学习
- 卷积神经网络

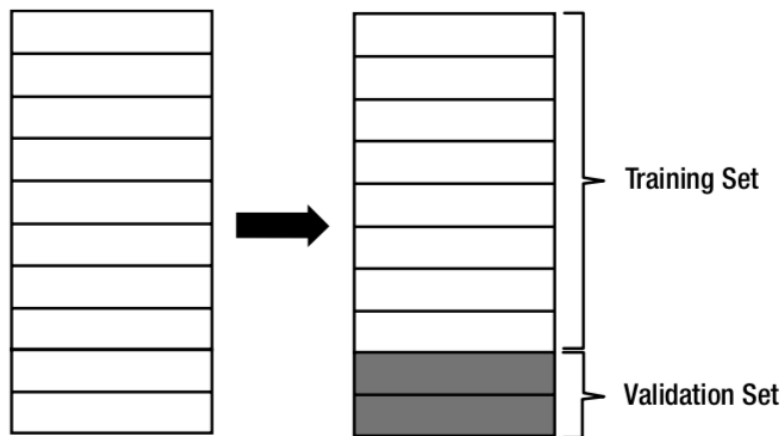


- 机器学习基本流程

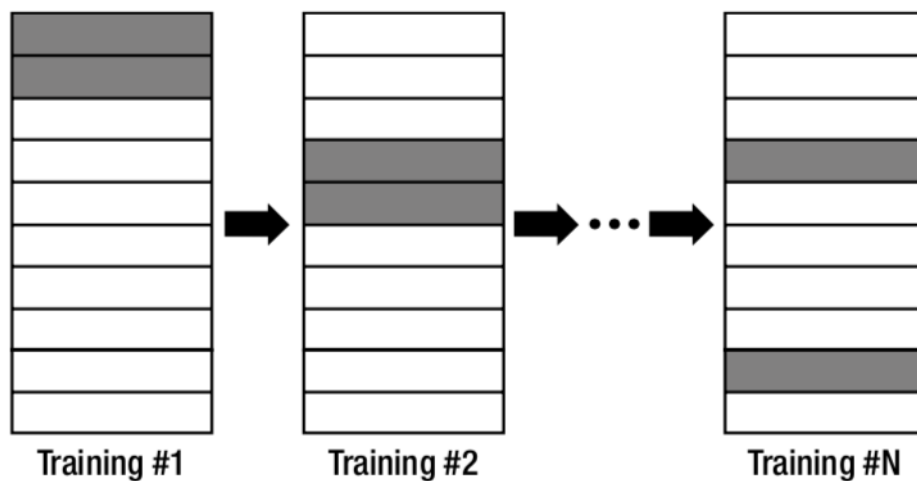


- 过拟合：解决方案

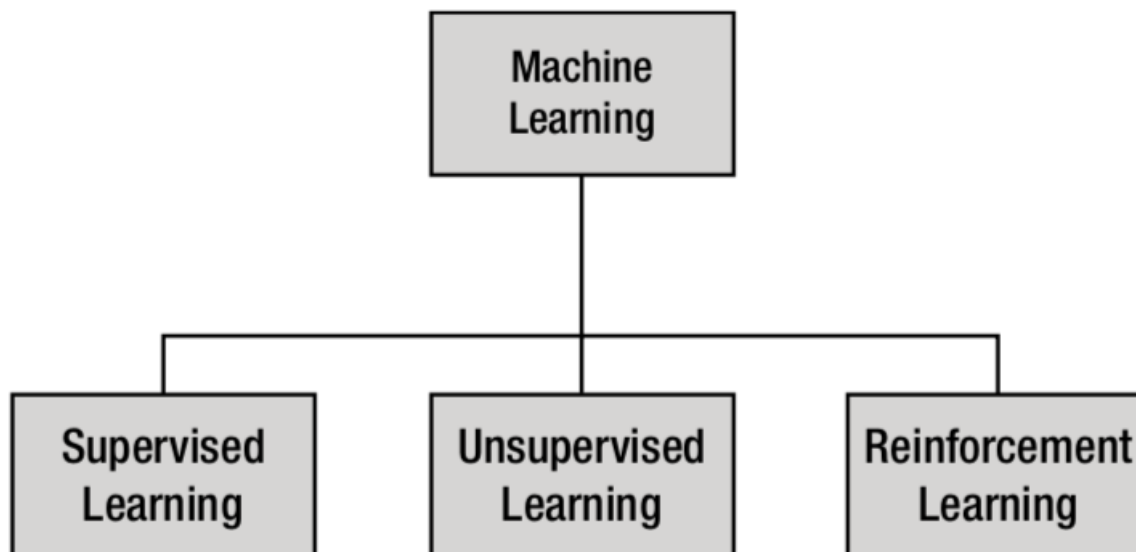
训练数据进一步
分为训练集和验证集



交叉验证



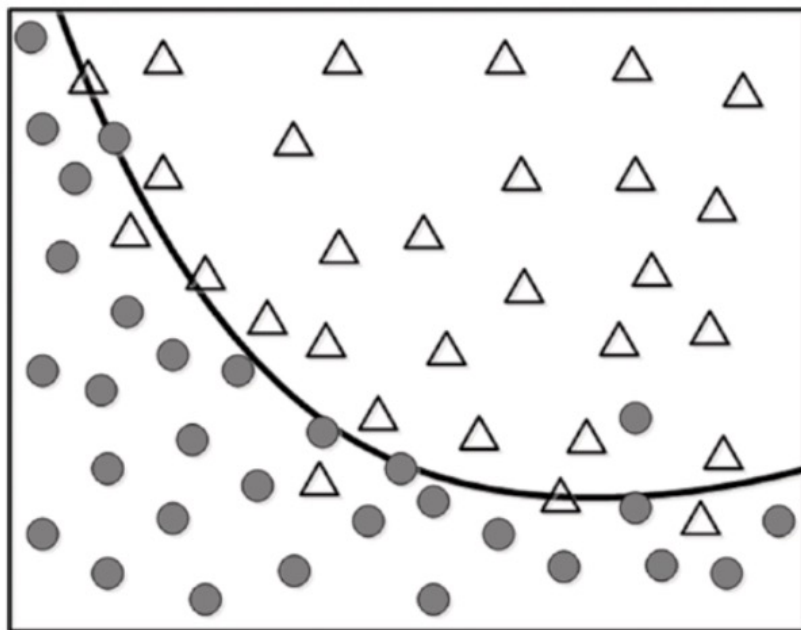
- 机器学习类型



Training Method	Training Data
Supervised Learning	{ input, correct output }
Unsupervised Learning	{ input }
Reinforced Learning	{ input, some output, grade for this output }

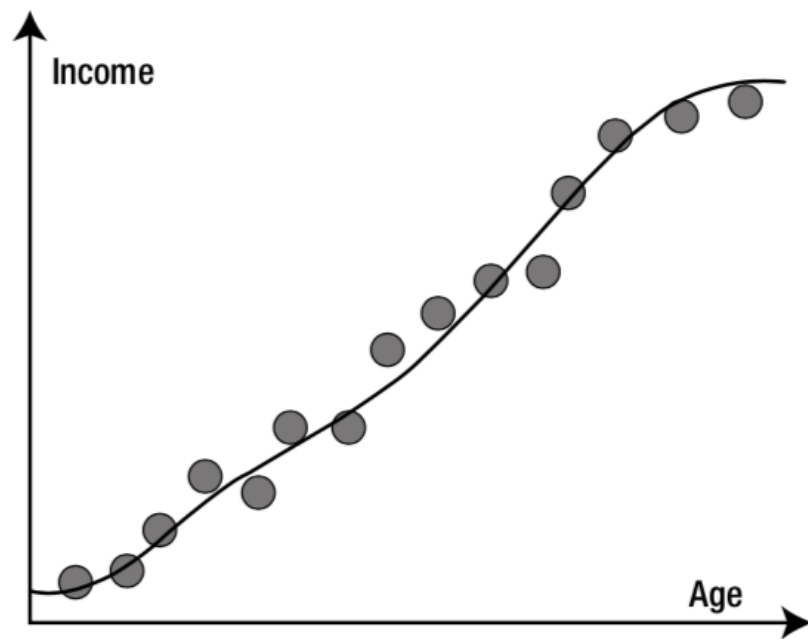
- 分类和回归：分类 $\{ \text{input}, \text{class} \}$

输入数据



$\{ X_1, Y_1, \triangle \}$
$\{ X_2, Y_2, \bullet \}$
...
$\{ X_N, Y_N, \bullet \}$

- 分类和回归：回归



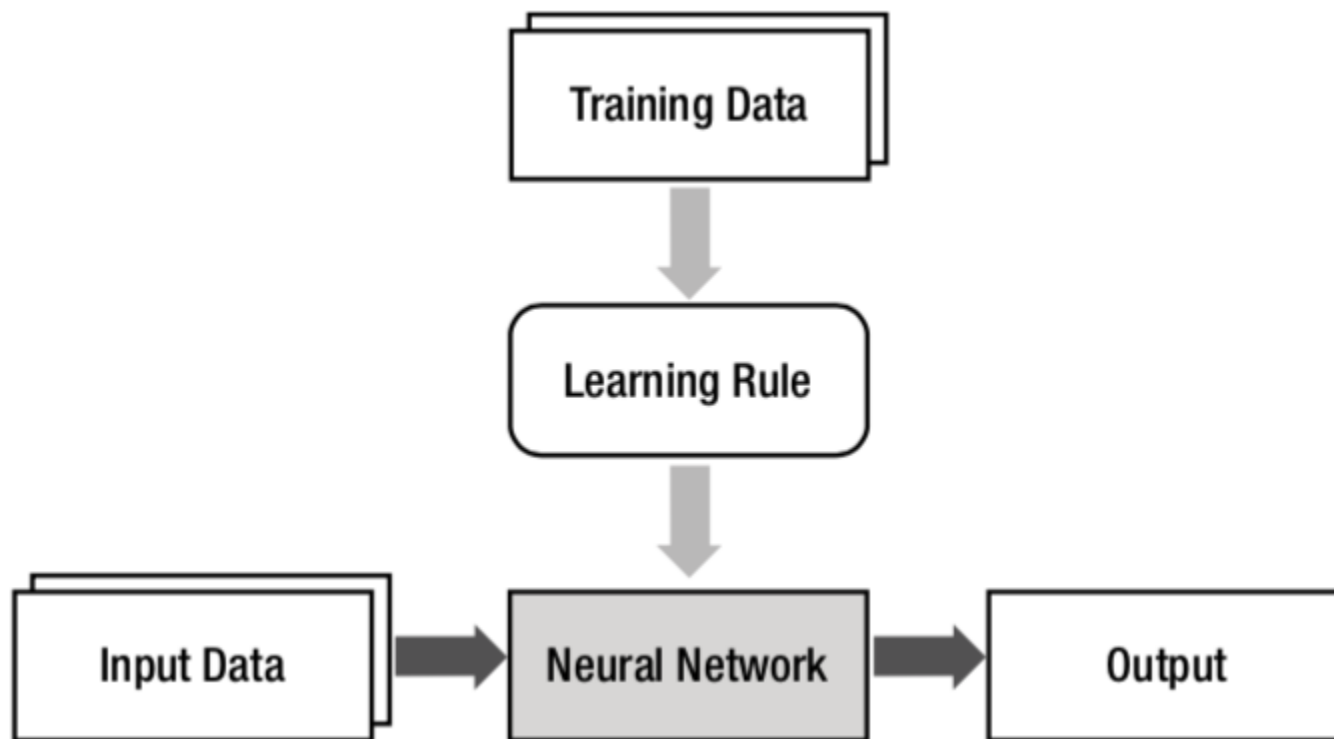
输入数据

$\{X_1, Y_1\}$
$\{X_2, Y_2\}$
...
$\{X_N, Y_N\}$

主要内容

- 机器学习
- 神经网络
- 训练多层神经网络
- 用于分类的多层神经网络
- 深度学习
- 卷积神经网络

- 用神经网络做机器学习的基本流程



- 和人脑的类比关系

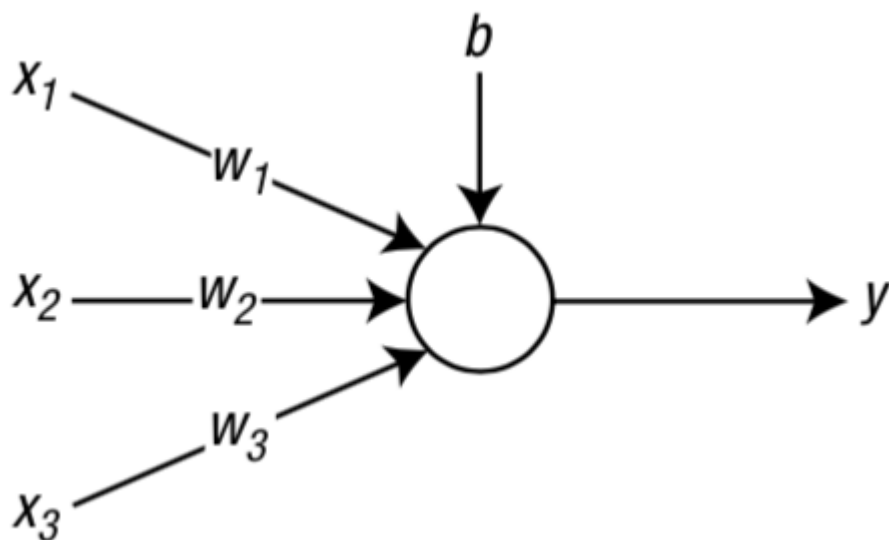
Brain	Neural Network
Neuron	Node
Connection of neurons	Connection weight

神经网络

- 和人脑的类比关系

Brain	Neural Network
Neuron	Node
Connection of neurons	Connection weight

- 基本运算



1. 加权和

$$v = (w_1 \times x_1) + (w_2 \times x_2) + (w_3 \times x_3) + b$$

或矢量化表示为

$$v = wx + b$$

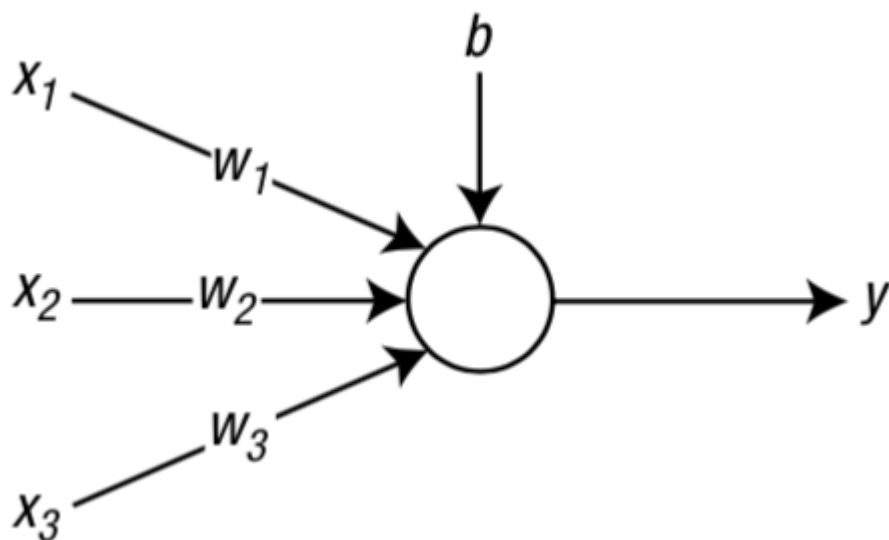
其中 $w = [w_1 \quad w_2 \quad w_3]$ $x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$

神经网络

- 和人脑的类比关系

Brain	Neural Network
Neuron	Node
Connection of neurons	Connection weight

- 基本运算



1. 加权和

$$v = (w_1 \times x_1) + (w_2 \times x_2) + (w_3 \times x_3) + b$$

或矢量化表示为

$$v = wx + b$$

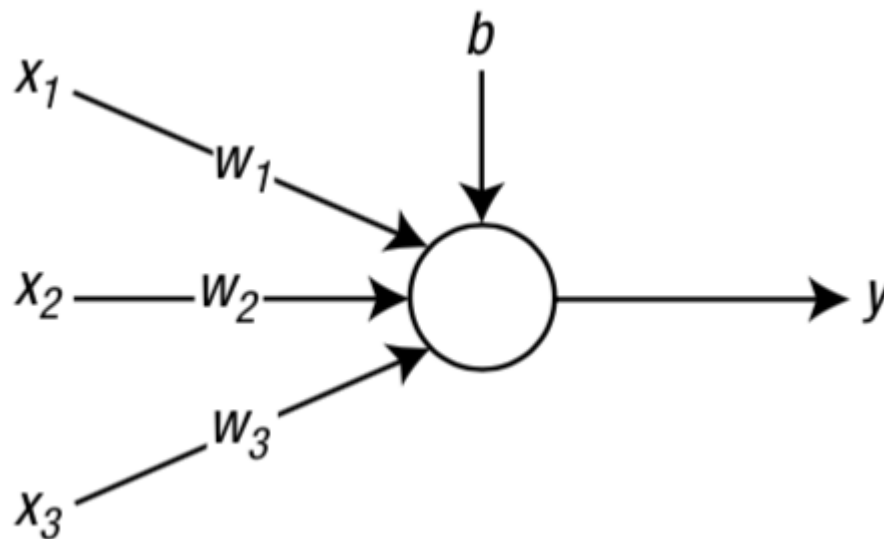
其中 $w = [w_1 \quad w_2 \quad w_3]$ $x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$

2. 激活

$$y = \varphi(v)$$

神经网络

- 基本运算



- 加权和

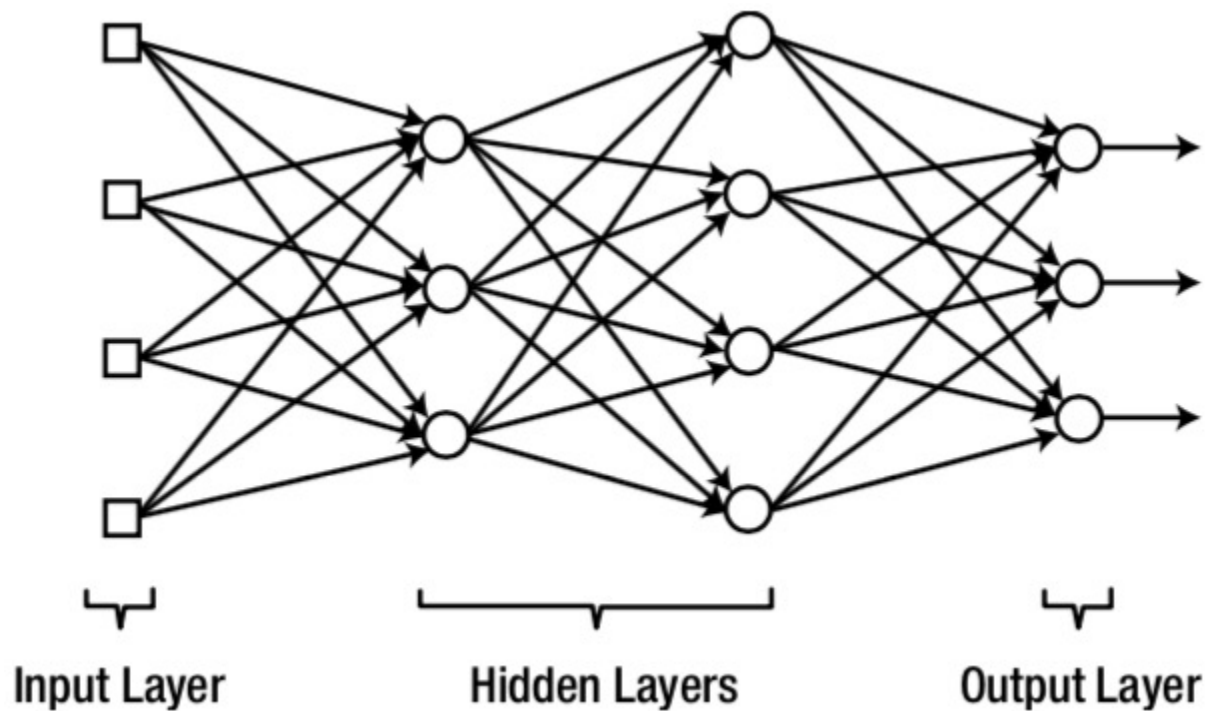
$$\begin{aligned}v &= w_1x_1 + w_2x_2 + w_3x_3 + b \\ &= wx + b\end{aligned}$$

- 激活

$$y = \varphi(v) = \varphi(wx + b)$$

神经网络

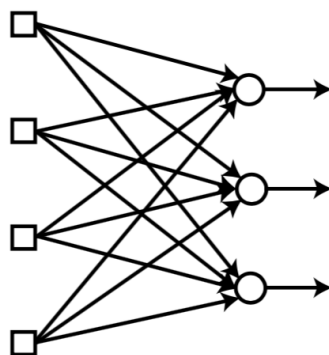
- 分层



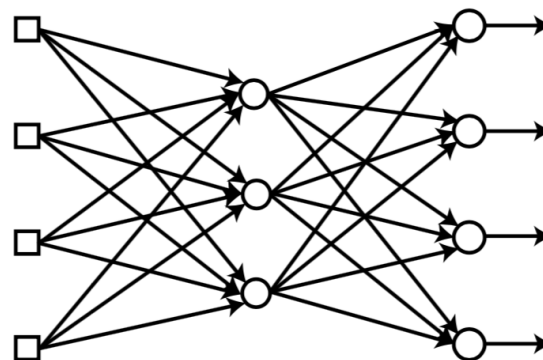
神经网络

- 分层

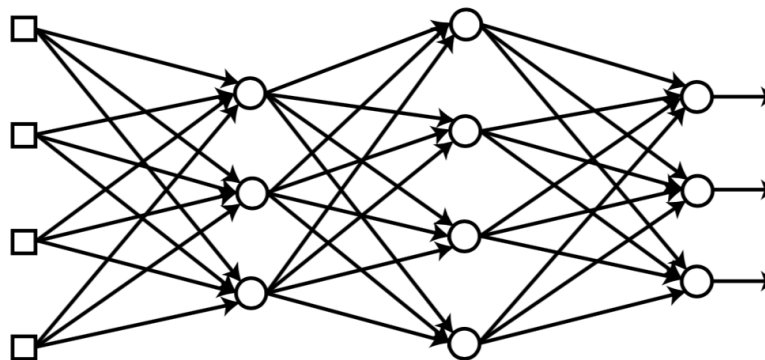
Single-Layer Neural Network		Input Layer - Output Layer
Multi-Layer Neural Network	Shallow Neural Network	Input Layer - Hidden Layer - Output Layer
	Deep Neural Network	Input Layer - Hidden Layers - Output Layers



Single-layer Neural Network

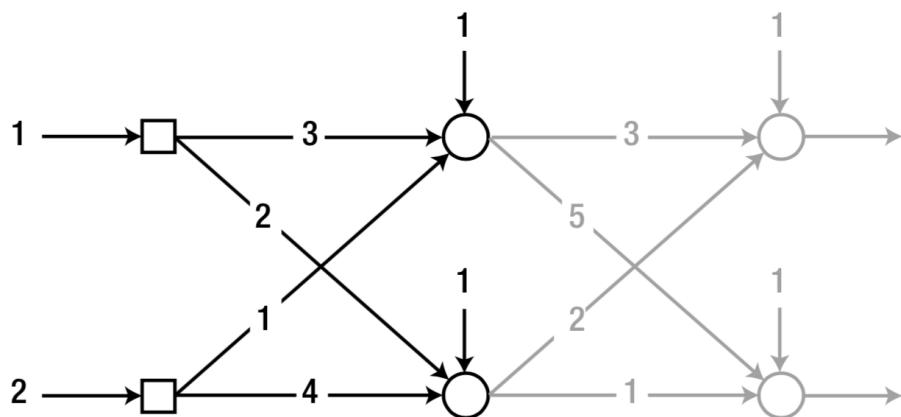
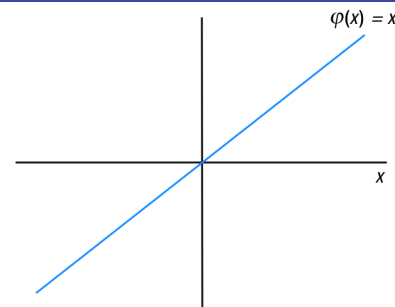


(Shallow) Multi-layer Neural Network

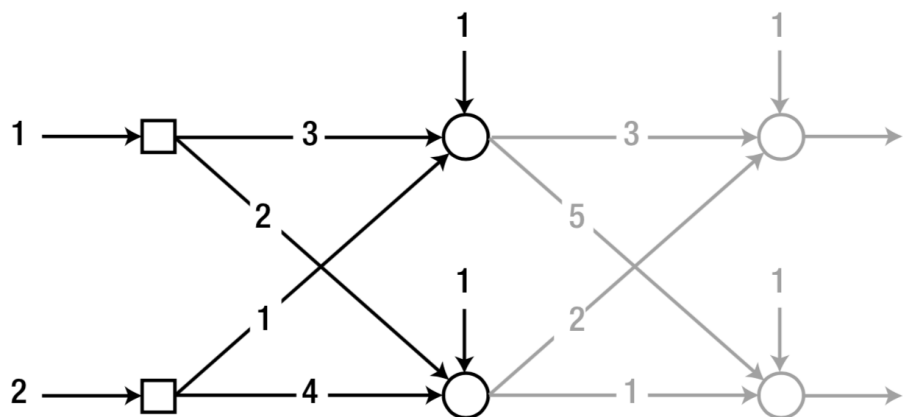
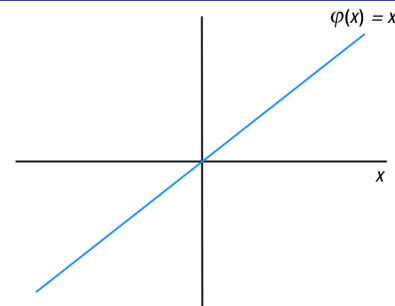


Deep Neural Network

- 计算方法（以线性激活函数为例）



- 计算方法（以线性激活函数为例）



Weighted sum: $v = (3 \times 1) + (1 \times 2) + 1 = 6$

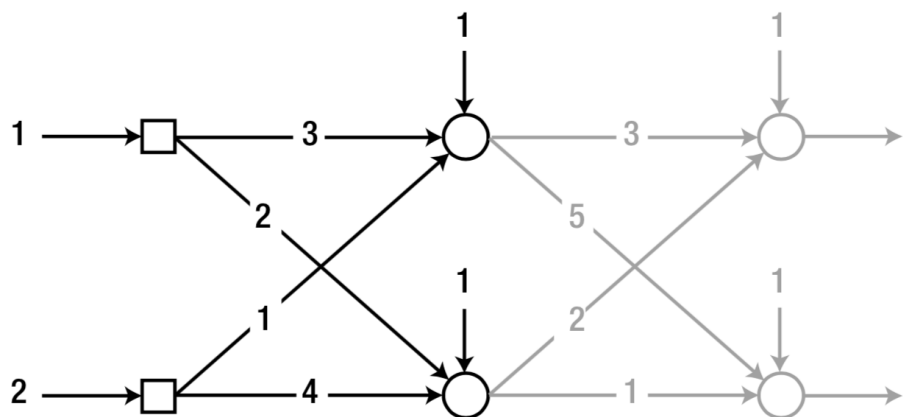
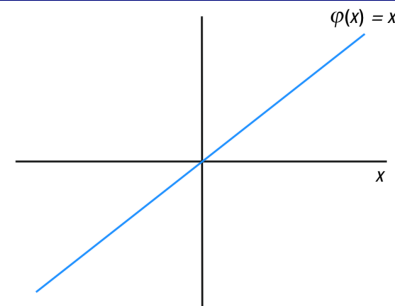
Output: $y = \varphi(v) = v = 6$

Weighted sum: $v = (2 \times 1) + (4 \times 2) + 1 = 11$

Output: $y = \varphi(v) = v = 11$

神经网络

- 计算方法（以线性激活函数为例）



$$\text{Weighted sum: } v = (3 \times 1) + (1 \times 2) + 1 = 6$$

$$\text{Output: } y = \varphi(v) = v = 6$$

$$\text{Weighted sum: } v = (2 \times 1) + (4 \times 2) + 1 = 11$$

$$\text{Output: } y = \varphi(v) = v = 11$$

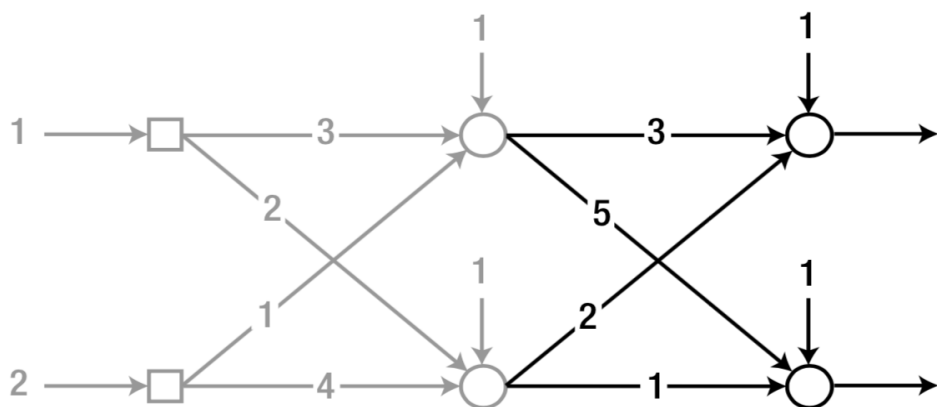
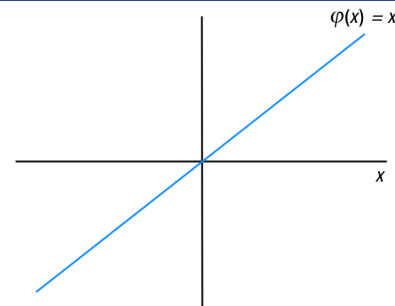
$$v = \begin{bmatrix} 3 \times 1 + 1 \times 2 + 1 \\ 2 \times 1 + 4 \times 2 + 1 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 6 \\ 11 \end{bmatrix}$$

$$v = Wx + b$$

$$W = \begin{bmatrix} \text{-- weights of the first node --} \\ \text{-- weights of the second node --} \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}$$

神经网络

- 计算方法（以线性激活函数为例）

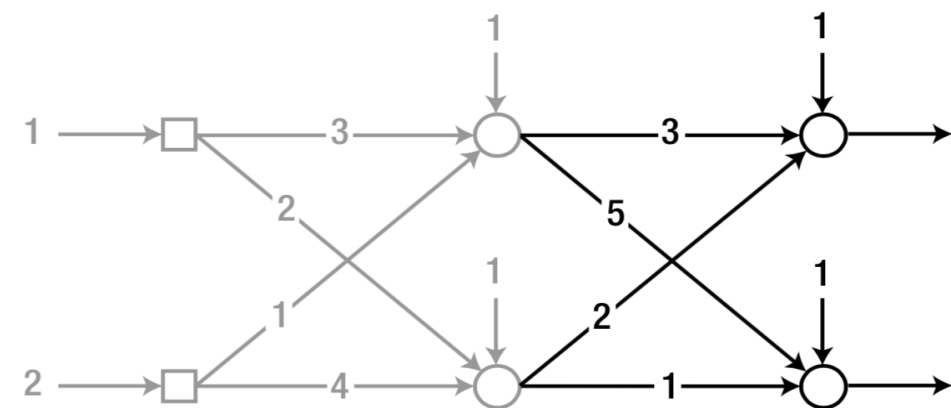
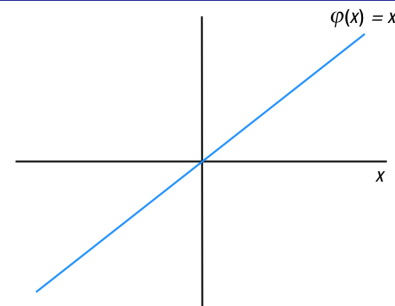


$$\text{Weighted sum: } v = \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 11 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 41 \\ 42 \end{bmatrix}$$

$$\text{Output: } y = \varphi(v) = v = \begin{bmatrix} 41 \\ 42 \end{bmatrix}$$

神经网络

- 计算方法（以线性激活函数为例）



$$\text{Weighted sum: } v = \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 11 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 41 \\ 42 \end{bmatrix}$$

$$\text{Output: } y = \varphi(v) = v = \begin{bmatrix} 41 \\ 42 \end{bmatrix}$$

$$v = \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 11 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

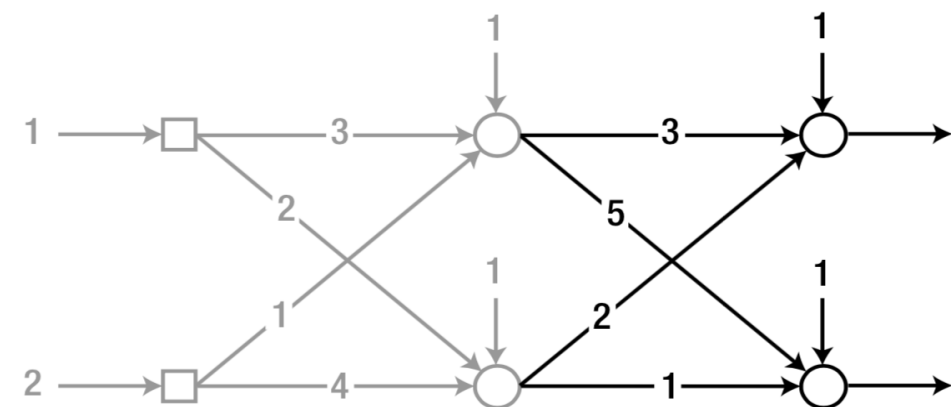
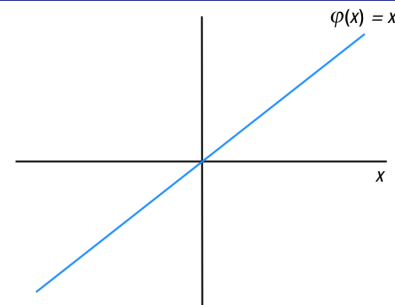
$$= \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \left(\begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) + \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 13 & 11 \\ 17 & 9 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 6 \\ 7 \end{bmatrix}$$

神经网络

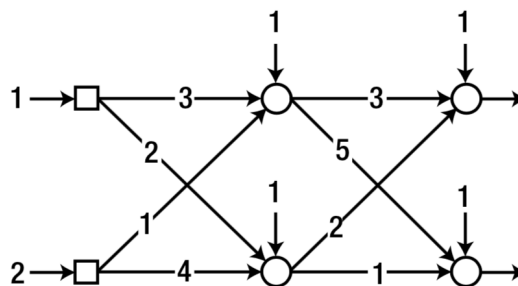
- 计算方法（以线性激活函数为例）



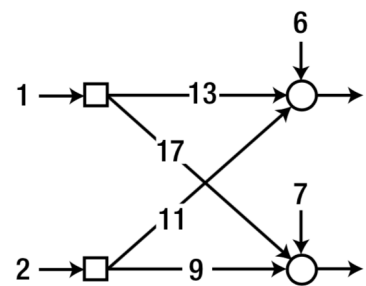
$$\text{Weighted sum: } v = \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 11 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 41 \\ 42 \end{bmatrix}$$

$$\text{Output: } y = \varphi(v) = v = \begin{bmatrix} 41 \\ 42 \end{bmatrix}$$

$$\begin{aligned} v &= \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 11 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \left(\begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 13 & 11 \\ 17 & 9 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 6 \\ 7 \end{bmatrix} \end{aligned}$$

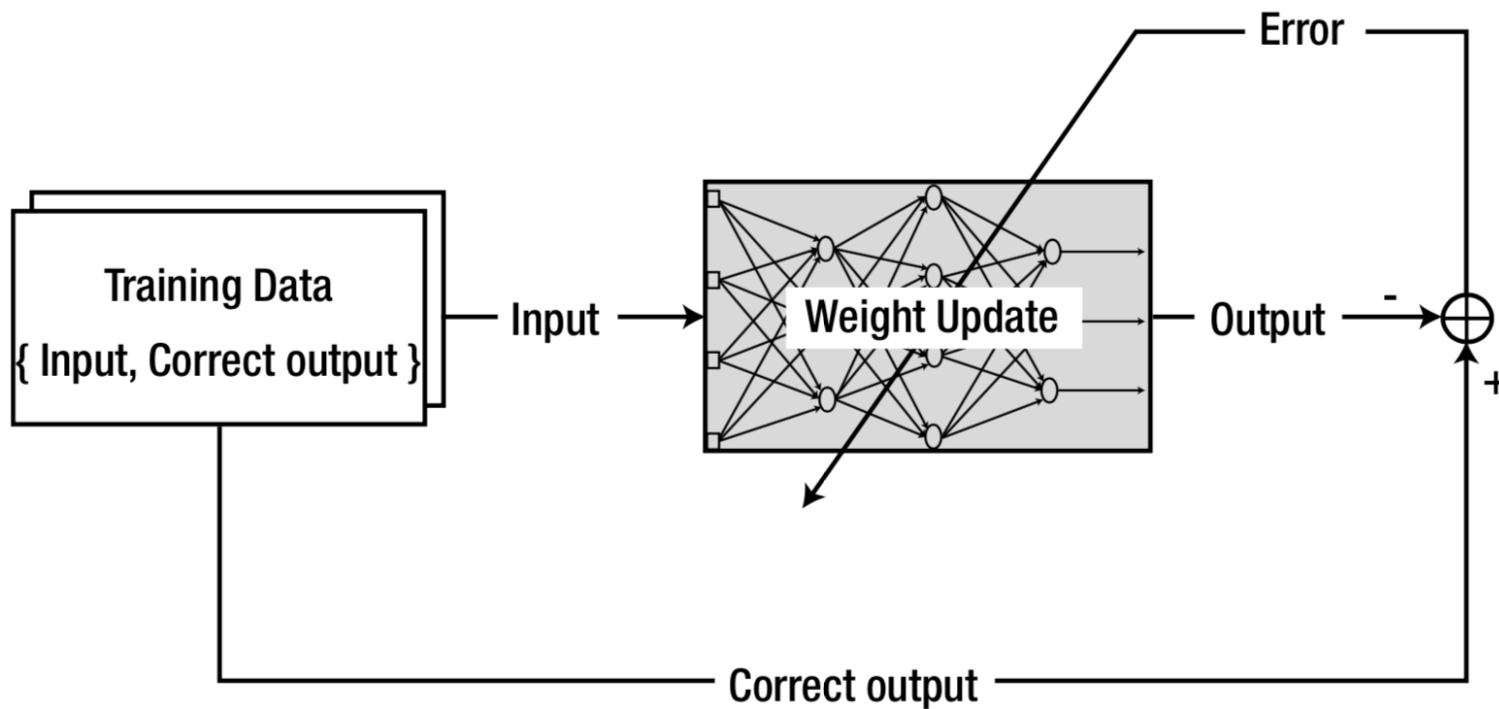


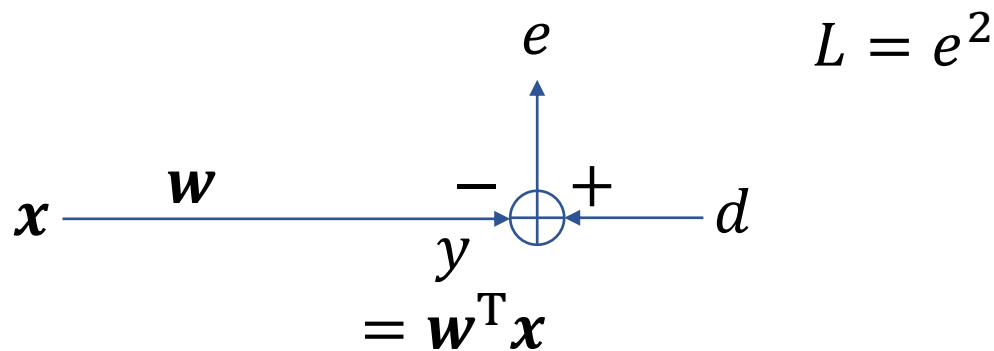
=



神经网络

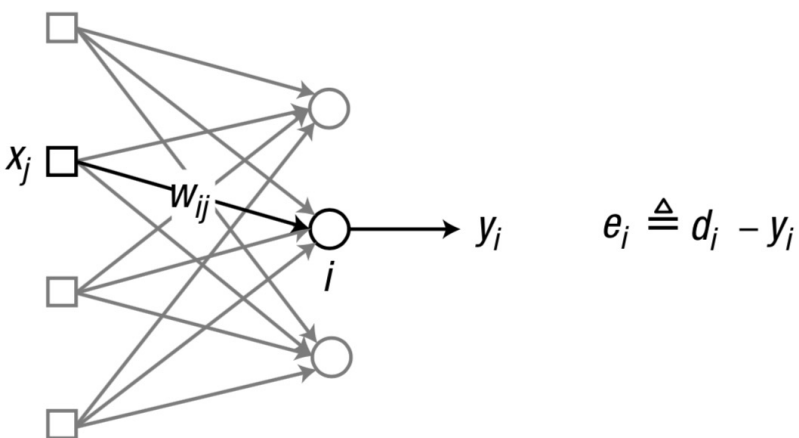
- 有监督学习





$$\begin{aligned}
 \mathbf{w}_{n+1} &= \mathbf{w}_n - \frac{\alpha}{2} \frac{\partial L}{\partial \mathbf{w}_n} \\
 &= \mathbf{w}_n - \frac{\alpha}{2} \frac{\partial L}{\partial e} \frac{\partial e}{\partial y} \frac{\partial y}{\partial \mathbf{w}_n} \\
 &= \mathbf{w}_n - \frac{\alpha}{2} (2e)(-1)(\mathbf{x}) \\
 &= \mathbf{w}_n + \alpha e \mathbf{x}
 \end{aligned}$$

- 训练单层神经网络：Delta规则



$$w_{ij} \leftarrow w_{ij} + \alpha e_i x_j$$

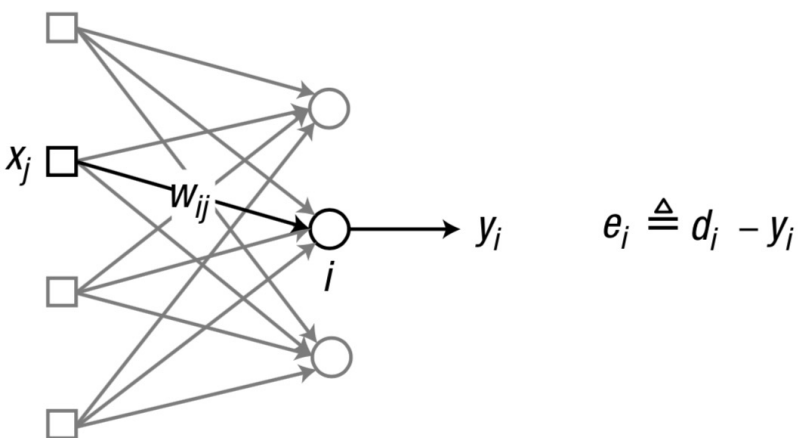
x_j = The output from the input node j , ($j = 1, 2, 3$)

e_i = The error of the output node i

w_{ij} = The weight between the output node i and input node j

α = Learning rate ($0 < \alpha \leq 1$)

- 训练单层神经网络: Delta规则



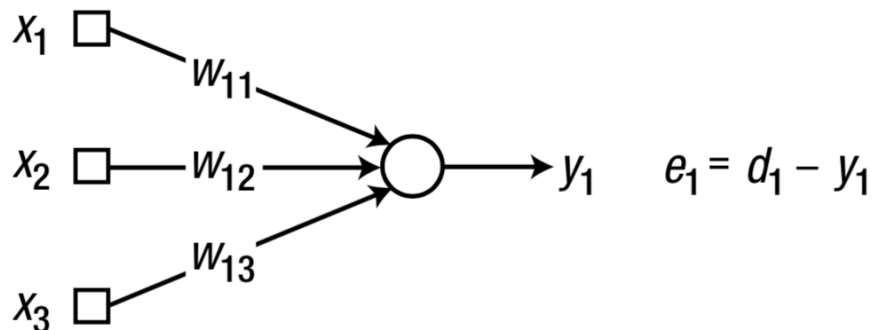
$$w_{ij} \leftarrow w_{ij} + \alpha e_i x_j$$

x_j = The output from the input node j , ($j = 1, 2, 3$)

e_i = The error of the output node i

w_{ij} = The weight between the output node i and input node j

α = Learning rate ($0 < \alpha \leq 1$)



$$w_{11} \leftarrow w_{11} + \alpha e_1 x_1$$

$$w_{12} \leftarrow w_{12} + \alpha e_1 x_2$$

$$w_{13} \leftarrow w_{13} + \alpha e_1 x_3$$

- 训练单层神经网络：Delta规则

1. 初始化权重

2. 将输入值送入神经网络，计算输出值，以及和正确输出值的误差

$$e_i = d_i - y_i$$

3. 计算权重更新量

$$\Delta w_{ij} = \alpha e_i x_j$$

4. 更新权重

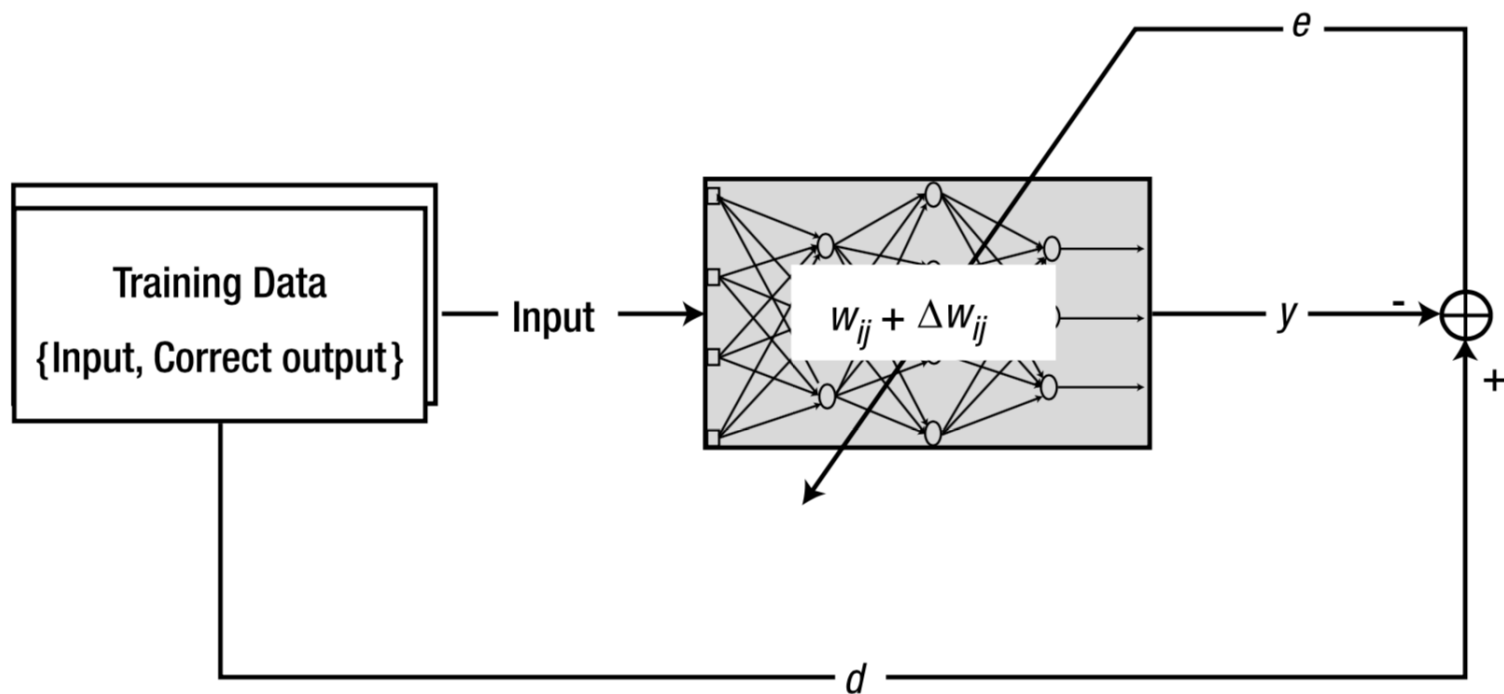
$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$

5. 对所有训练数据执行步骤2-4

6. 重复步骤2-5直到误差小于给定阈值

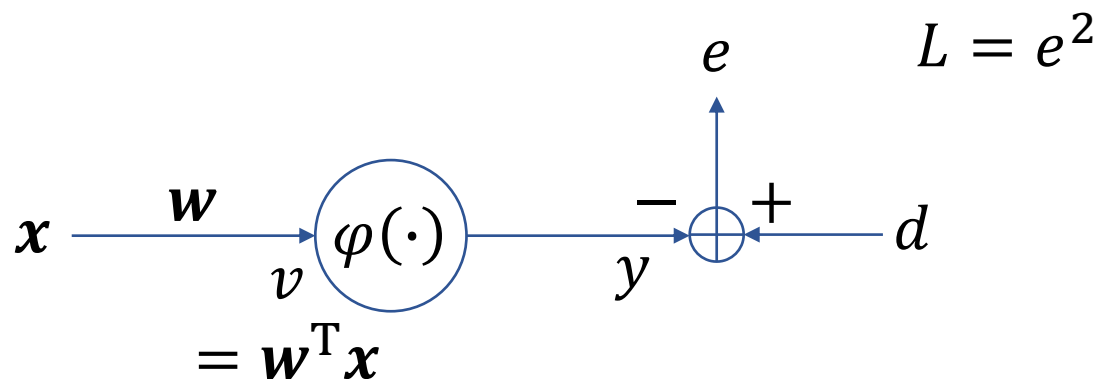
神经网络

- 训练单层神经网络：Delta规则

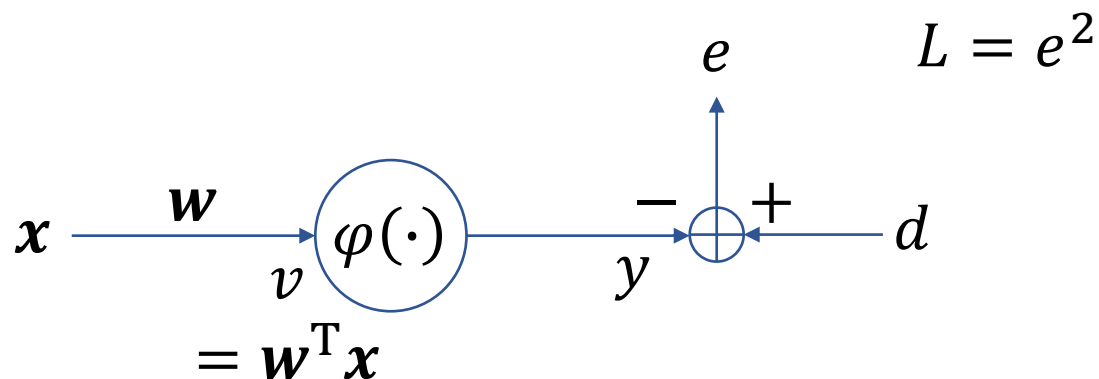


- 广义Delta规则 $w_{ij} \leftarrow w_{ij} + \alpha \delta_i x_j$

其中 $\delta_i = \varphi'(v_i)e_i$



$$\begin{aligned}
 \mathbf{w}_{n+1} &= \mathbf{w}_n - \frac{\alpha}{2} \frac{\partial L}{\partial \mathbf{w}_n} \\
 &= \mathbf{w}_n - \frac{\alpha}{2} \frac{\partial L}{\partial e} \frac{\partial e}{\partial y} \frac{\partial y}{\partial v} \frac{\partial v}{\partial \mathbf{w}_n} \\
 &= \mathbf{w}_n - \frac{\alpha}{2} (2e)(-1) \varphi'(v)(\mathbf{x}) \\
 &= \mathbf{w}_n + \alpha \varphi'(v) e \mathbf{x} \\
 &= \mathbf{w}_n + \alpha \delta \mathbf{x}
 \end{aligned}$$



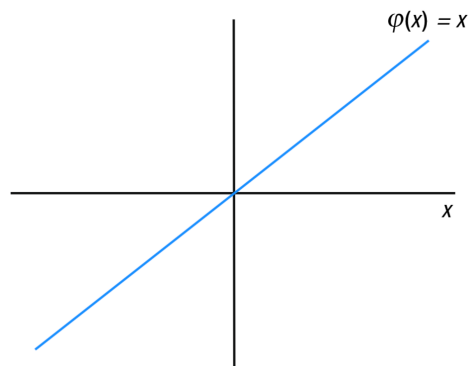
$$\begin{aligned}
 \mathbf{w}_{n+1} &= \mathbf{w}_n - \frac{\alpha}{2} \frac{\partial L}{\partial \mathbf{w}_n} \\
 &= \mathbf{w}_n - \frac{\alpha}{2} \frac{\partial L}{\partial e} \frac{\partial e}{\partial y} \frac{\partial y}{\partial v} \frac{\partial v}{\partial \mathbf{w}_n} \\
 &= \mathbf{w}_n - \frac{\alpha}{2} (2e)(-1)\varphi'(v)(\mathbf{x}) \\
 &= \mathbf{w}_n + \alpha \varphi'(v) e \mathbf{x} \\
 &= \mathbf{w}_n + \alpha \delta \mathbf{x}
 \end{aligned}$$

- 广义Delta规则 $w_{ij} \leftarrow w_{ij} + \alpha \delta_i x_j$

其中

$$\delta_i = \varphi'(v_i) e_i$$

- 对于线性激活函数



$$\delta_i = e_i$$
$$w_{ij} \leftarrow w_{ij} + \alpha e_i x_j$$

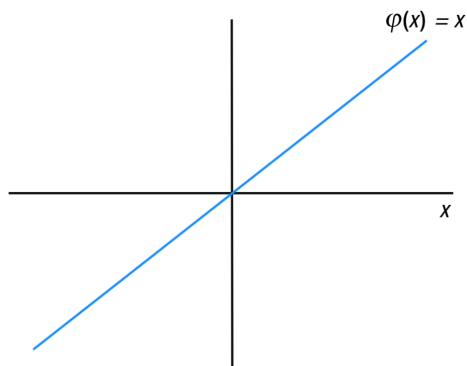
神经网络

- 广义Delta规则 $w_{ij} \leftarrow w_{ij} + \alpha \delta_i x_j$

其中

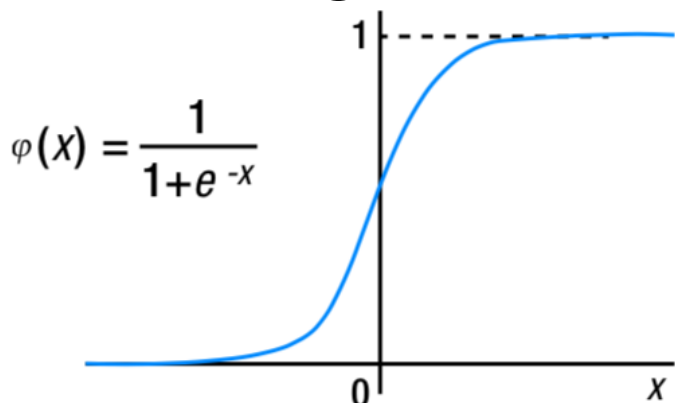
$$\delta_i = \varphi'(v_i) e_i$$

- 对于线性激活函数



$$\delta_i = e_i$$
$$w_{ij} \leftarrow w_{ij} + \alpha e_i x_j$$

- 对于Sigmoid激活函数

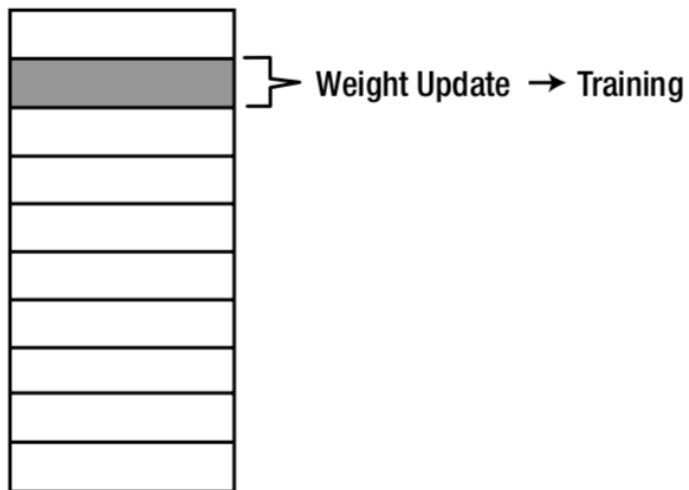


$$\varphi'(x) = \varphi(x)(1-\varphi(x))$$
$$\delta_i = \varphi'(v_i) e_i = \varphi(v_i)(1-\varphi(v_i)) e_i$$
$$w_{ij} \leftarrow w_{ij} + \alpha \varphi(v_i)(1-\varphi(v_i)) e_i x_j$$

$$\varphi(v) = \frac{1}{1 + e^{-v}} \quad \Rightarrow$$

$$\delta = \varphi'(v)e = \frac{-e^{-v}}{(1 + e^{-v})^2} e = \varphi(v)(1 - \varphi(v))e$$

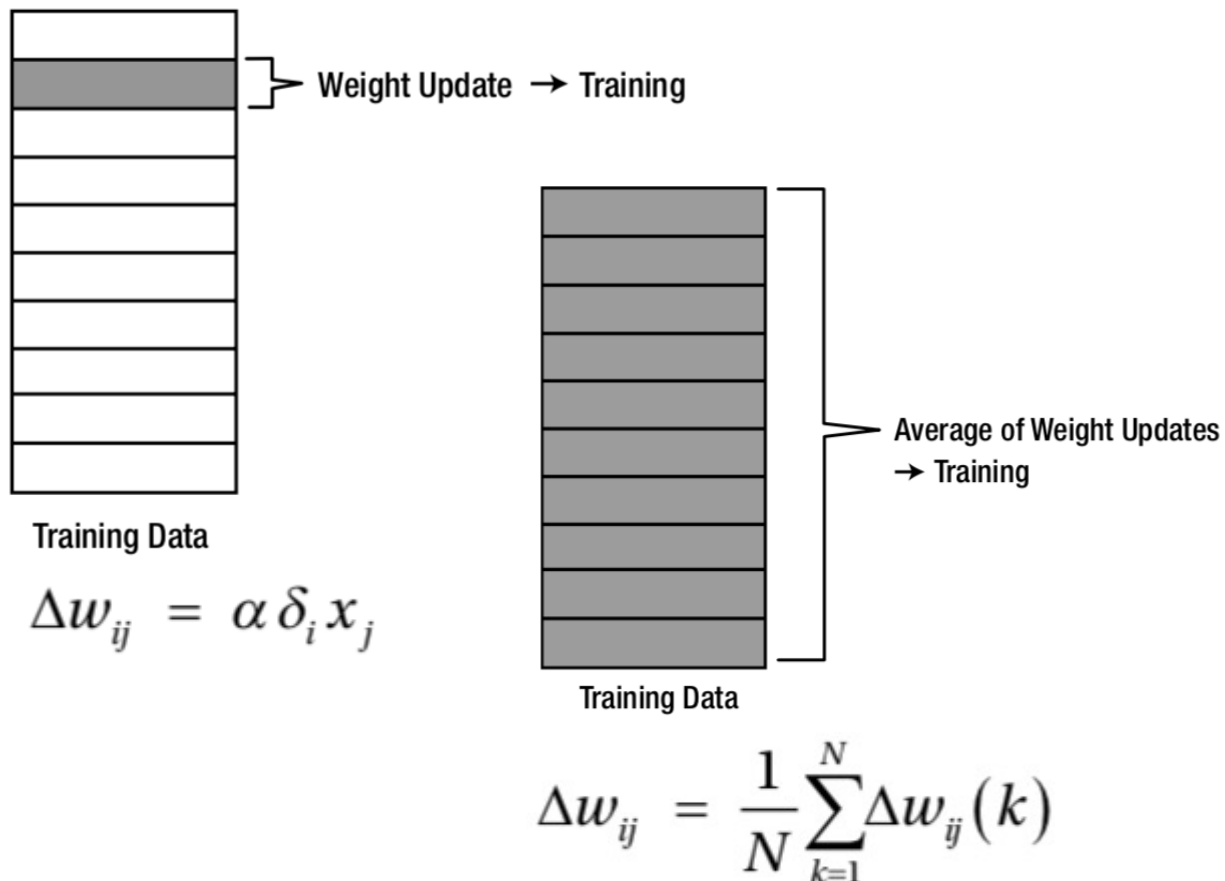
- SGD、Batch和Mini Batch



Training Data

$$\Delta w_{ij} = \alpha \delta_i x_j$$

- SGD、Batch和Mini Batch

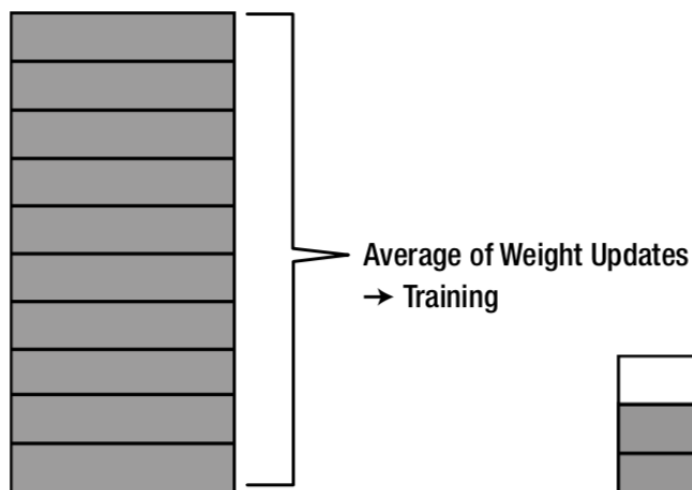


- SGD、Batch和Mini Batch



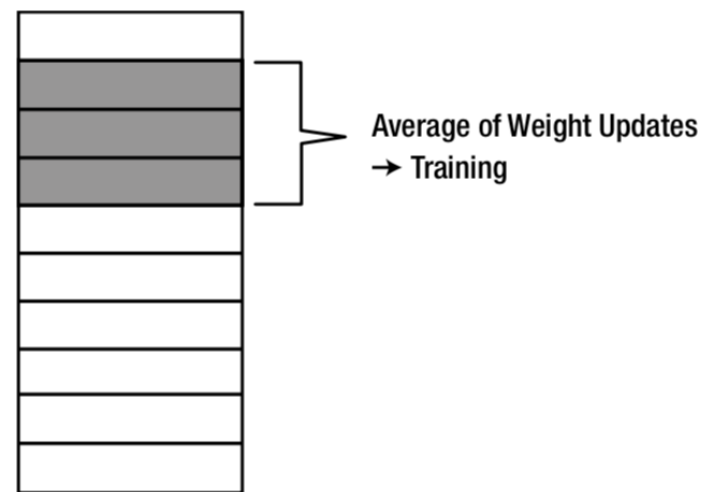
Training Data

$$\Delta w_{ij} = \alpha \delta_i x_j$$



Training Data

$$\Delta w_{ij} = \frac{1}{N} \sum_{k=1}^N \Delta w_{ij}(k)$$



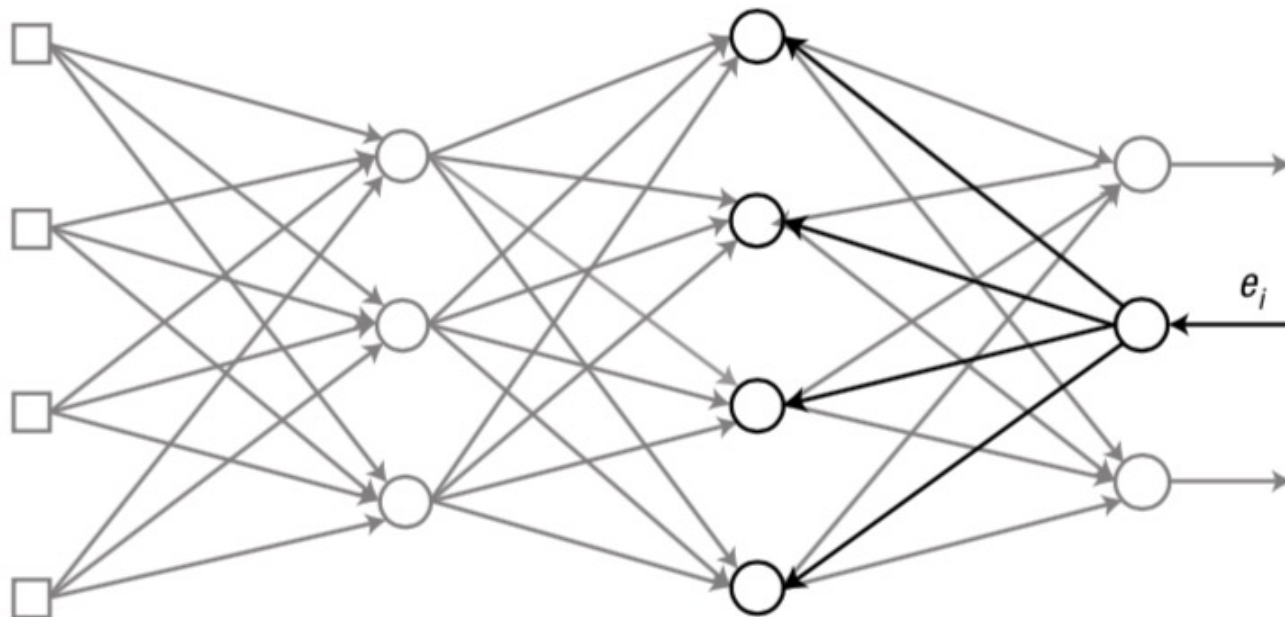
Training Data

主要内容

- 机器学习
- 神经网络
- 训练多层神经网络
- 用于分类的多层神经网络
- 深度学习
- 卷积神经网络

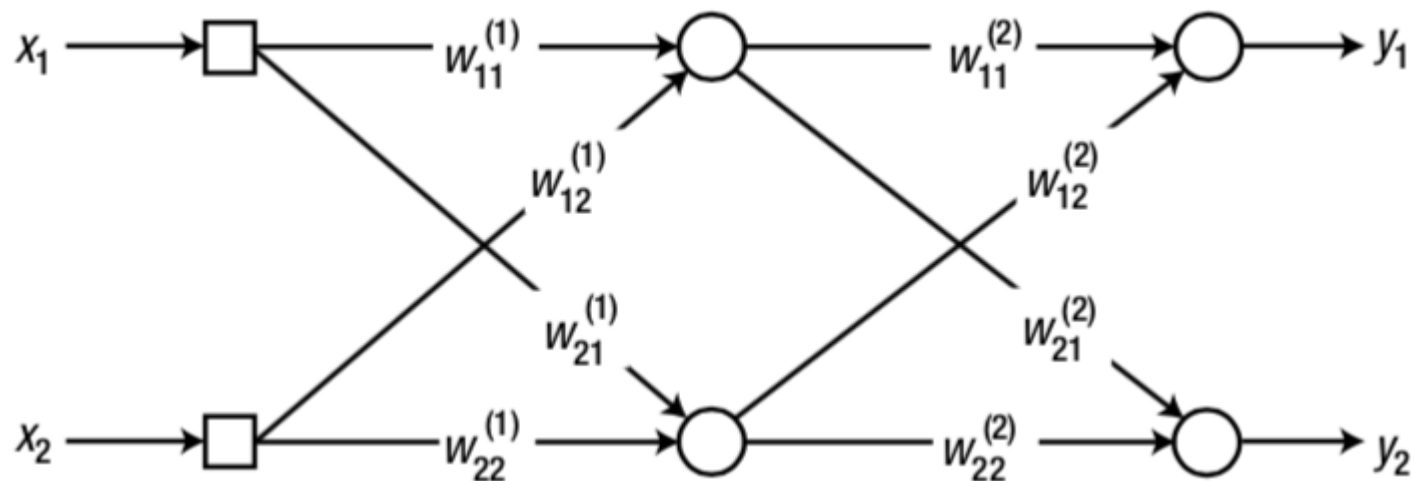
训练多层神经网络

- 后向传播



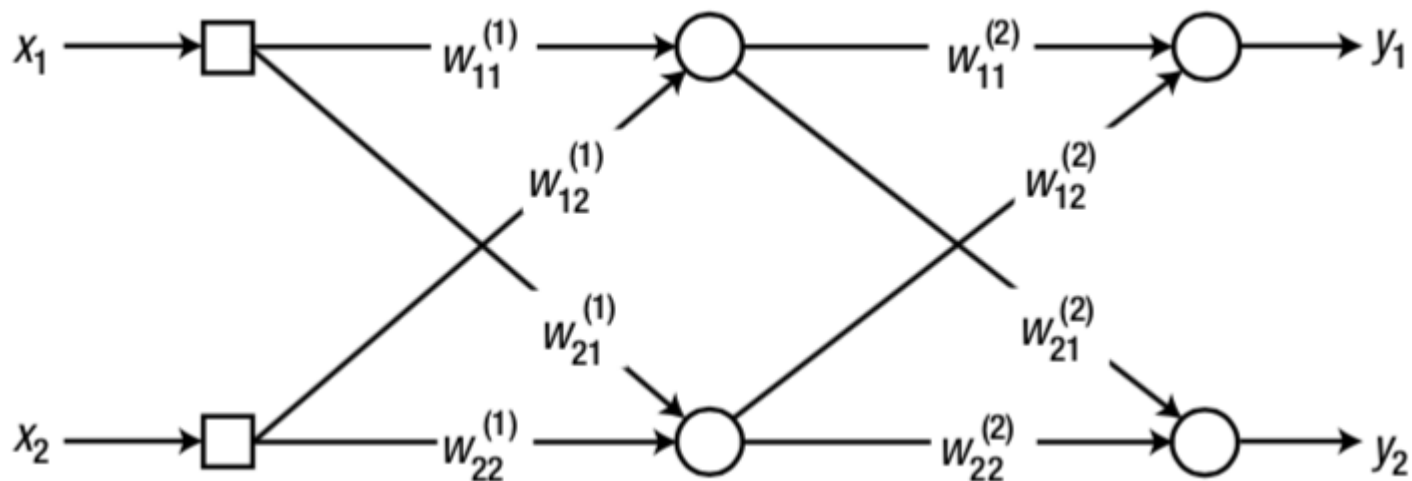
训练多层神经网络

- 前向计算过程



训练多层神经网络

- 前向计算过程

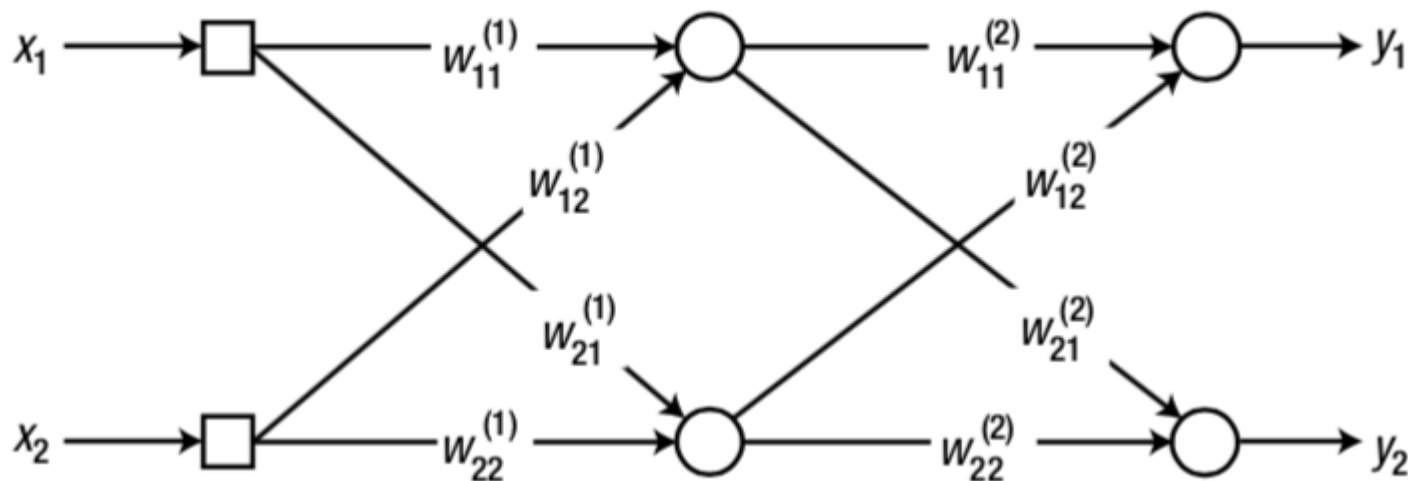


$$\begin{bmatrix} v_1^{(1)} \\ v_2^{(1)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \triangleq W_1 x$$

$$\begin{bmatrix} y_1^{(1)} \\ y_2^{(1)} \end{bmatrix} = \begin{bmatrix} \varphi(v_1^{(1)}) \\ \varphi(v_2^{(1)}) \end{bmatrix}$$

训练多层神经网络

- 前向计算过程

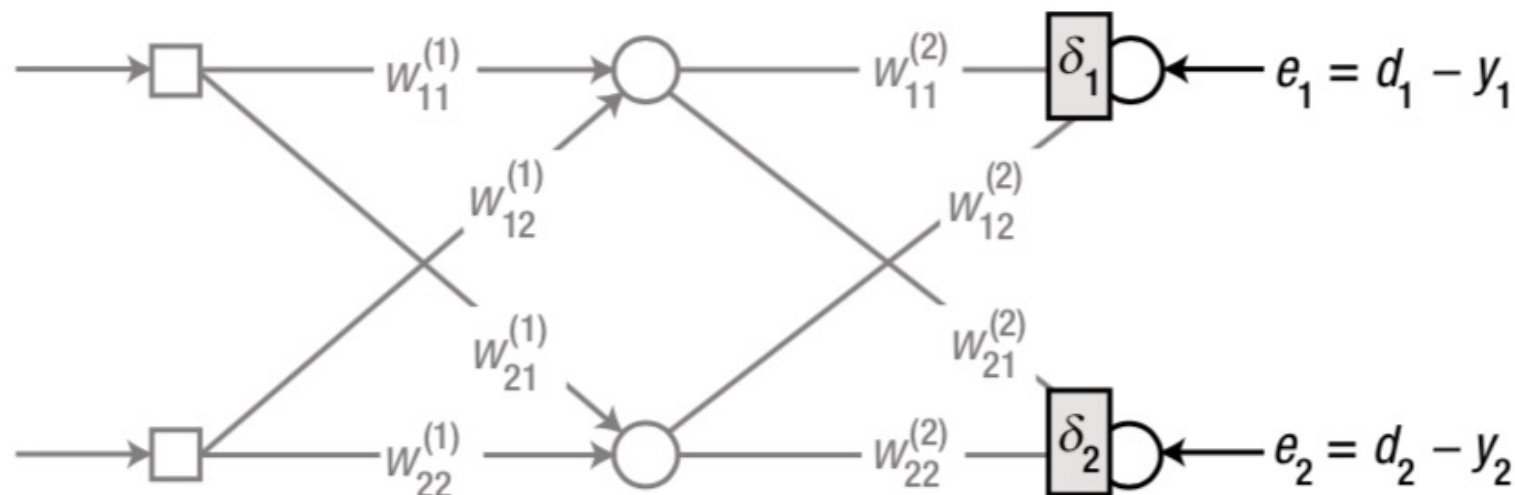


$$\begin{bmatrix} v_1^{(1)} \\ v_2^{(1)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \triangleq W_1 x \quad \begin{bmatrix} y_1^{(1)} \\ y_2^{(1)} \end{bmatrix} = \begin{bmatrix} \varphi(v_1^{(1)}) \\ \varphi(v_2^{(1)}) \end{bmatrix}$$

$$\begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} \end{bmatrix} \begin{bmatrix} y_1^{(1)} \\ y_2^{(1)} \end{bmatrix} \triangleq W_2 y^{(1)} \quad \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} \varphi(v_1) \\ \varphi(v_2) \end{bmatrix}$$

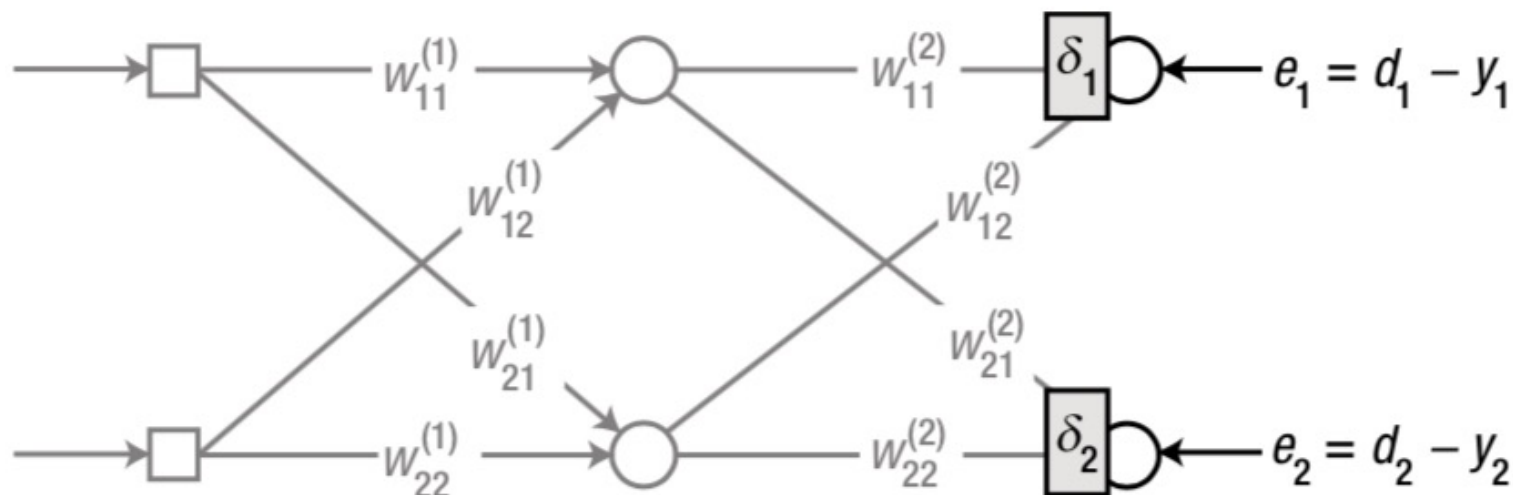
训练多层神经网络

- 后向传播算法



训练多层神经网络

- 后向传播算法



$$e_1 = d_1 - y_1$$

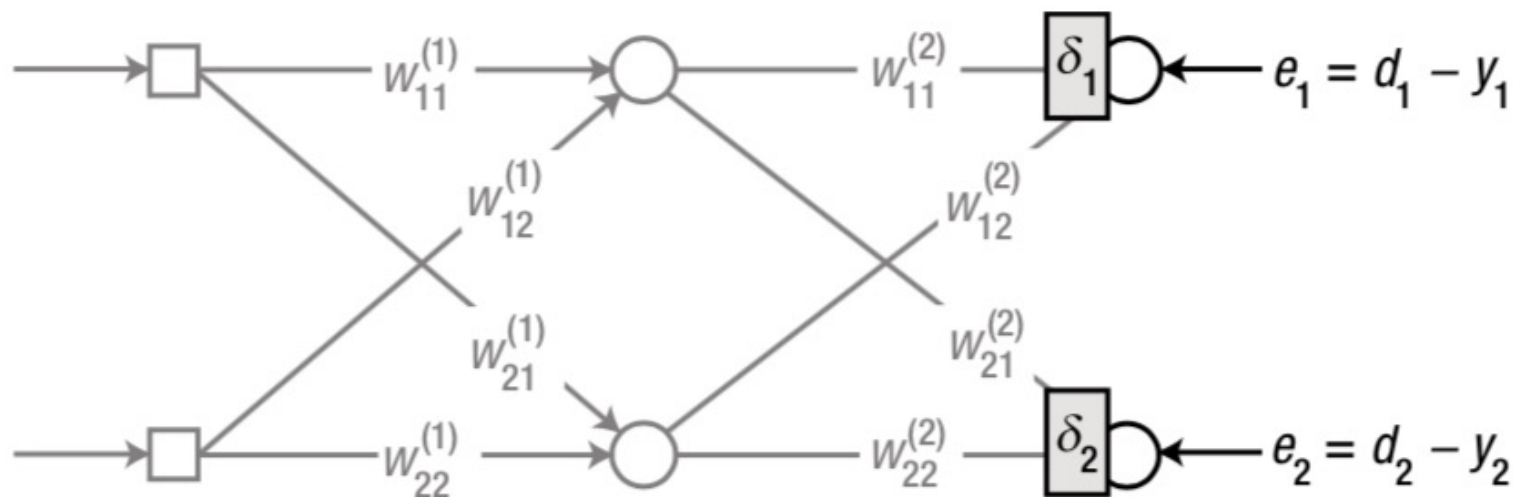
$$\delta_1 = \varphi'(v_1)e_1$$

$$e_2 = d_2 - y_2$$

$$\delta_2 = \varphi'(v_2)e_2$$

训练多层神经网络

- 后向传播算法

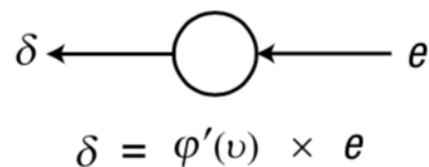
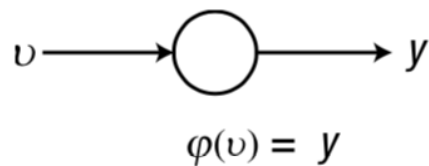


$$e_1 = d_1 - y_1$$

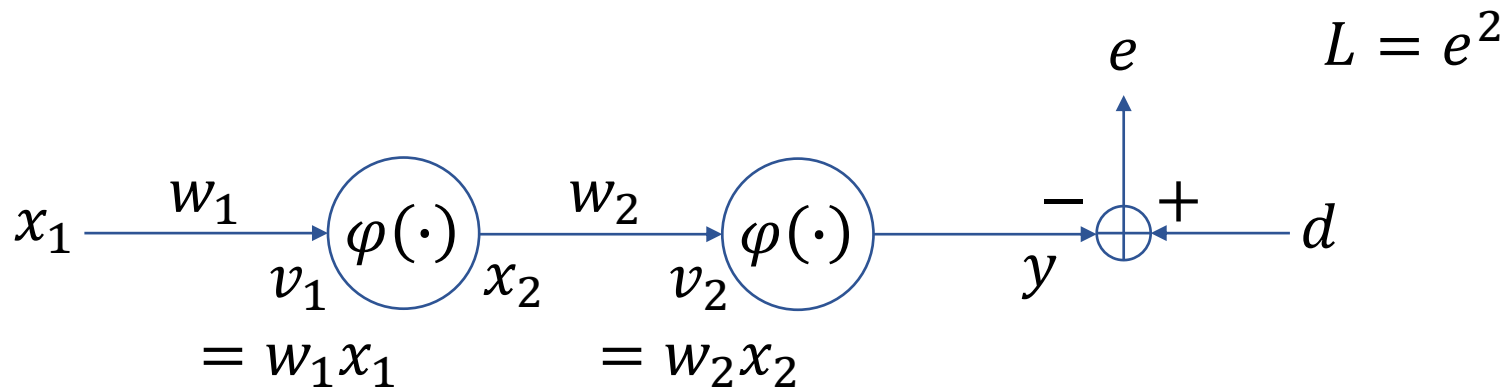
$$\delta_1 = \varphi'(v_1)e_1$$

$$e_2 = d_2 - y_2$$

$$\delta_2 = \varphi'(v_2)e_2$$



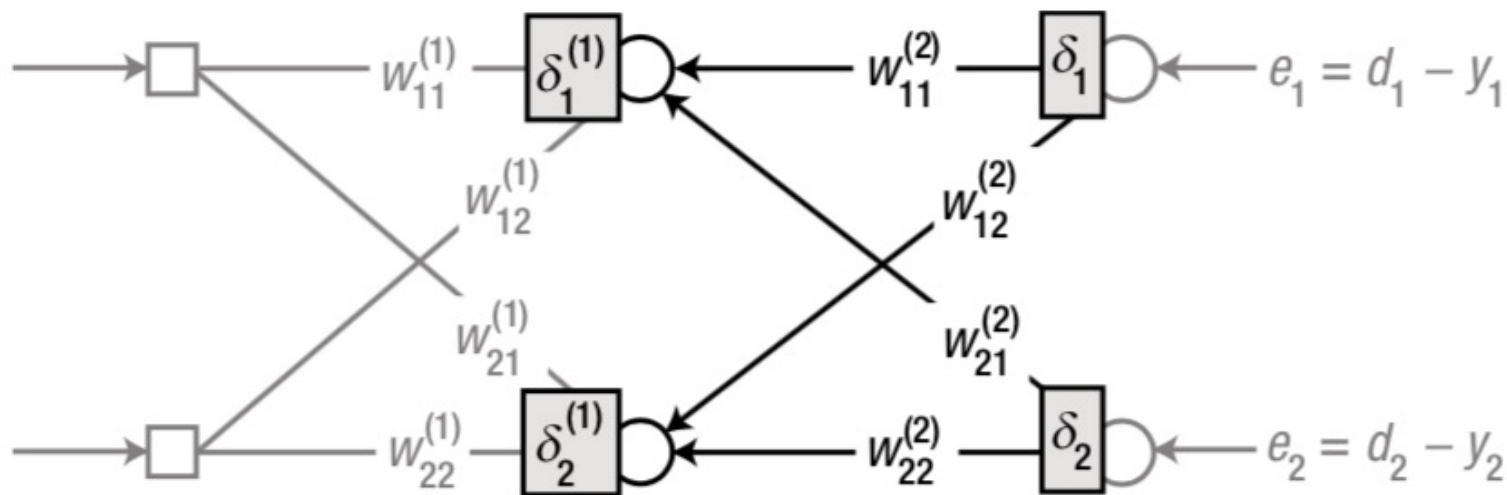
训练多层神经网络



$$\begin{aligned}
 \frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial y} \cdot \frac{\partial y}{\partial v_2} \cdot \frac{\partial v_2}{\partial x_2} \cdot \frac{\partial x_2}{\partial v_1} \cdot \frac{\partial v_1}{\partial w_1} \\
 &= -2 \cdot e \cdot \varphi'(v_2) \cdot w_2 \cdot \varphi'(v_1) \cdot x_1 \\
 &= -2 \cdot \boxed{\delta_2} \cdot w_2 \cdot \varphi'(v_1) \cdot x_1 \\
 &= -2 \cdot \boxed{e_1} \cdot \varphi'(v_1) \cdot x_1 \\
 &= -2 \cdot \underline{\delta_1} \cdot x_1
 \end{aligned}$$

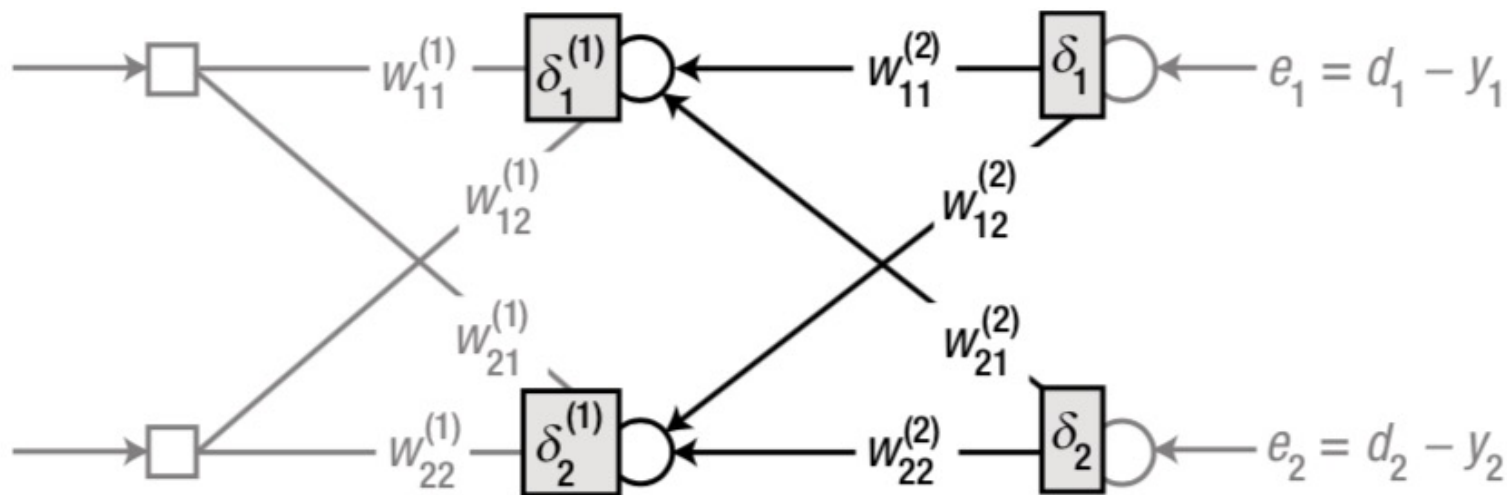
训练多层神经网络

- 后向传播算法



训练多层神经网络

- 后向传播算法



$$e_1^{(1)} = w_{11}^{(2)} \delta_1 + w_{21}^{(2)} \delta_2$$

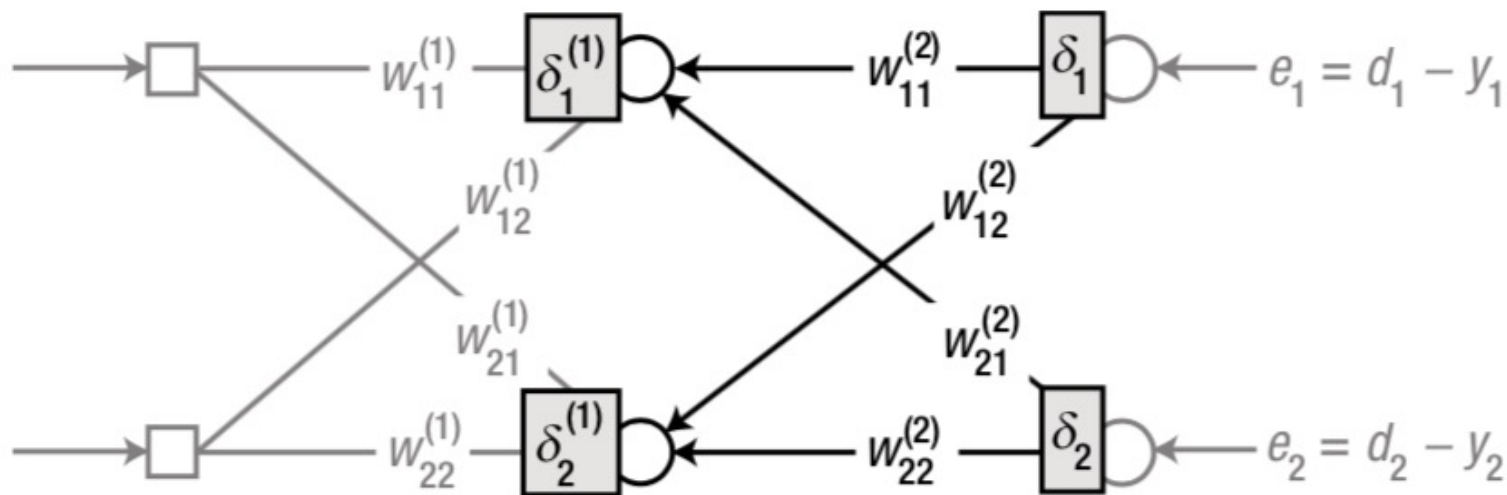
$$\delta_1^{(1)} = \varphi'(v_1^{(1)}) e_1^{(1)}$$

$$e_2^{(1)} = w_{12}^{(2)} \delta_1 + w_{22}^{(2)} \delta_2$$

$$\delta_2^{(1)} = \varphi'(v_2^{(1)}) e_2^{(1)}$$

训练多层神经网络

- 后向传播算法



$$e_1^{(1)} = w_{11}^{(2)} \delta_1 + w_{21}^{(2)} \delta_2$$

$$\delta_1^{(1)} = \varphi'(v_1^{(1)}) e_1^{(1)}$$

$$e_2^{(1)} = w_{12}^{(2)} \delta_1 + w_{22}^{(2)} \delta_2$$

$$\delta_2^{(1)} = \varphi'(v_2^{(1)}) e_2^{(1)}$$

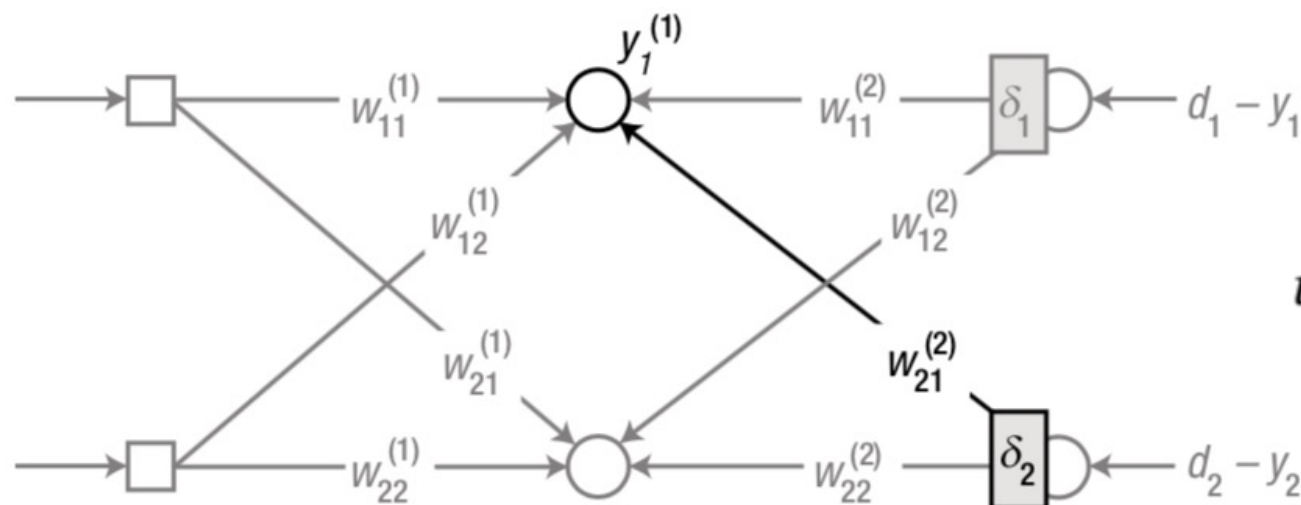
$$\begin{bmatrix} e_1^{(1)} \\ e_2^{(1)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(2)} & w_{21}^{(2)} \\ w_{12}^{(2)} & w_{22}^{(2)} \end{bmatrix} \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix} = W_2^T \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix}$$

训练多层神经网络

- 后向传播算法

$$\Delta w_{ij} = \alpha \delta_i x_j$$

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$



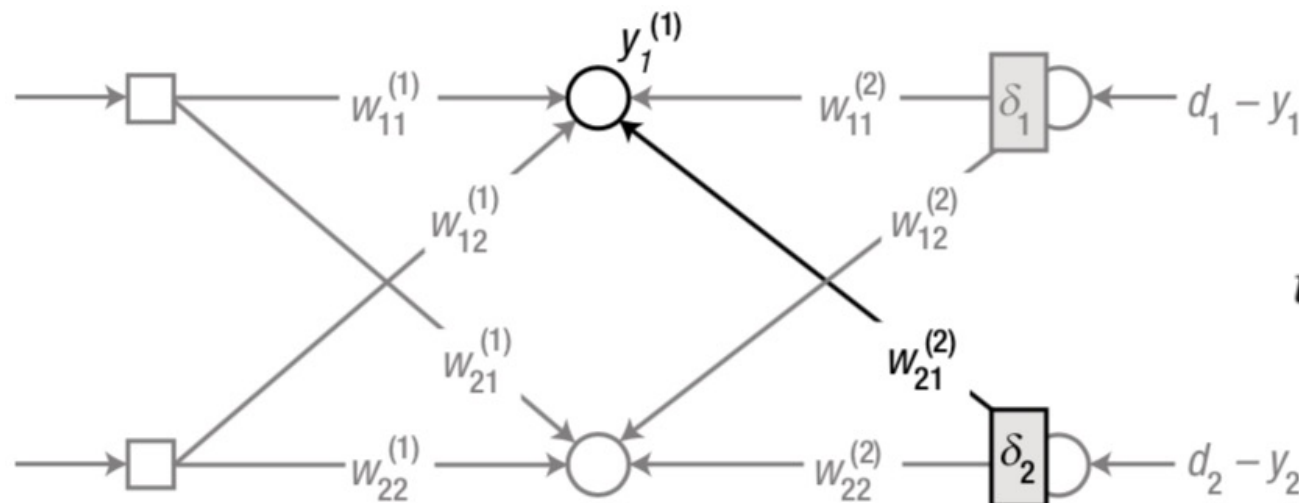
$$w_{21}^{(2)} \leftarrow w_{21}^{(2)} + \alpha \delta_2 y_1^{(1)}$$

训练多层神经网络

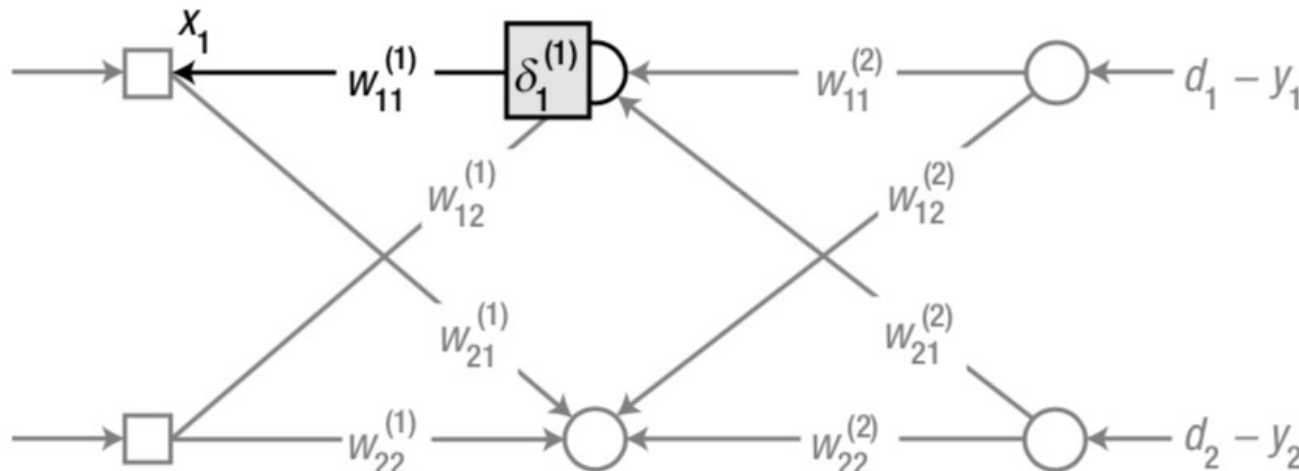
- 后向传播算法

$$\Delta w_{ij} = \alpha \delta_i x_j$$

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$



$$w_{21}^{(2)} \leftarrow w_{21}^{(2)} + \alpha \delta_2 y_1^{(1)}$$



$$w_{11}^{(1)} \leftarrow w_{11}^{(1)} + \alpha \delta_1^{(1)} x_1$$

训练多层神经网络

- 后向传播算法步骤

1. 初始化权重

2. 将输入值送入神经网络，计算输出值，和正确输出值的误差，以及输出节点的Delta

$$e = d - y$$

$$\delta = \varphi'(v)e$$

3. 将输出节点的Delta后向传播，用于计算相邻节点的Delta

$$e^{(k)} = W^T \delta$$

$$\delta^{(k)} = \varphi'(v^{(k)})e^{(k)}$$

4. 重复步骤3直至到达输入节点之后的第一个隐含层

5. 根据以下规则更新权重

$$\Delta w_{ij} = \alpha \delta_i x_j$$

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$

6. 对所有训练数据执行步骤2-5

7. 重复步骤2-6直到误差小于给定阈值

训练多层神经网络

- 动量

$$\Delta w = \alpha \delta x$$

$$m = \Delta w + \beta m^-$$

$$w = w + m$$

$$m^- = m$$

训练多层神经网络

- 动量

$$\Delta w = \alpha \delta x$$

$$m = \Delta w + \beta m^-$$

$$w = w + m$$

$$m^- = m$$

$$m(0) = 0$$

$$m(1) = \Delta w(1) + \beta m(0) = \Delta w(1)$$

$$m(2) = \Delta w(2) + \beta m(1) = \Delta w(2) + \beta \Delta w(1)$$

$$\begin{aligned} m(3) &= \Delta w(3) + \beta m(2) = \Delta w(3) + \beta \{ \Delta w(2) + \beta \Delta w(1) \} \\ &= \Delta w(3) + \beta \Delta w(2) + \beta^2 \Delta w(1) \end{aligned}$$

$$\vdots$$

训练多层神经网络

- 动量

$$\Delta w = \alpha \delta x$$

$$m = \Delta w + \beta m^-$$

$$w = w + m$$

$$m^- = m$$

$$m(0) = 0$$

$$m(1) = \Delta w(1) + \beta m(0) = \Delta w(1)$$

$$m(2) = \Delta w(2) + \beta m(1) = \Delta w(2) + \beta \Delta w(1)$$

$$\begin{aligned} m(3) &= \Delta w(3) + \beta m(2) = \Delta w(3) + \beta \{ \Delta w(2) + \beta \Delta w(1) \} \\ &= \Delta w(3) + \beta \Delta w(2) + \beta^2 \Delta w(1) \end{aligned}$$

$\vdots$

- 参考上例

初始化

```
mmt1 = zeros(size(W1));  
mmt2 = zeros(size(W2));
```

更新权重

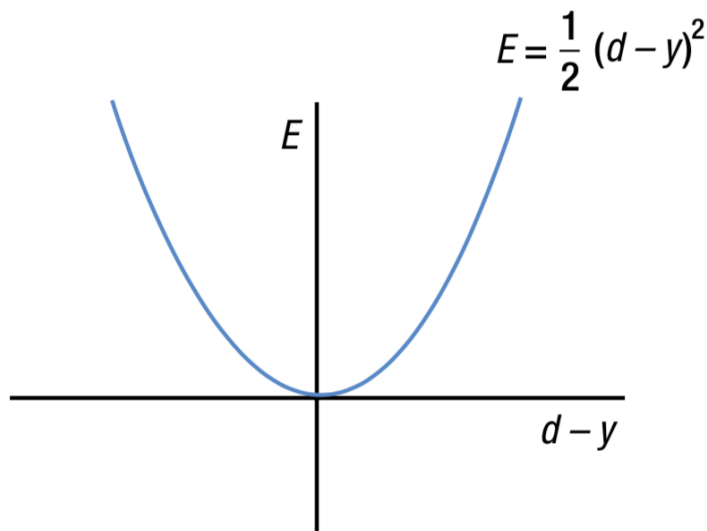
```
dw1  = alpha*delta1*x';  
mmt1 = dw1 + beta*mmt1;  
W1   = W1 + mmt1;  
  
dw2  = alpha*delta*y1';  
mmt2 = dw2 + beta*mmt2;  
W2   = W2 + mmt2;
```

训练多层神经网络

- 代价函数和学习规则

平方和
$$J = \sum_{i=1}^M \frac{1}{2} (d_i - y_i)^2$$

交叉熵
$$J = \sum_{i=1}^M \{-d_i \ln(y_i) - (1-d_i) \ln(1-y_i)\}$$



训练多层神经网络

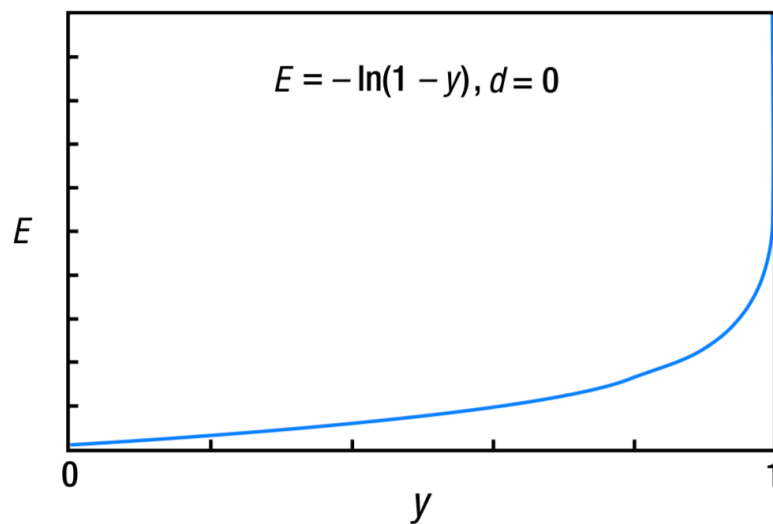
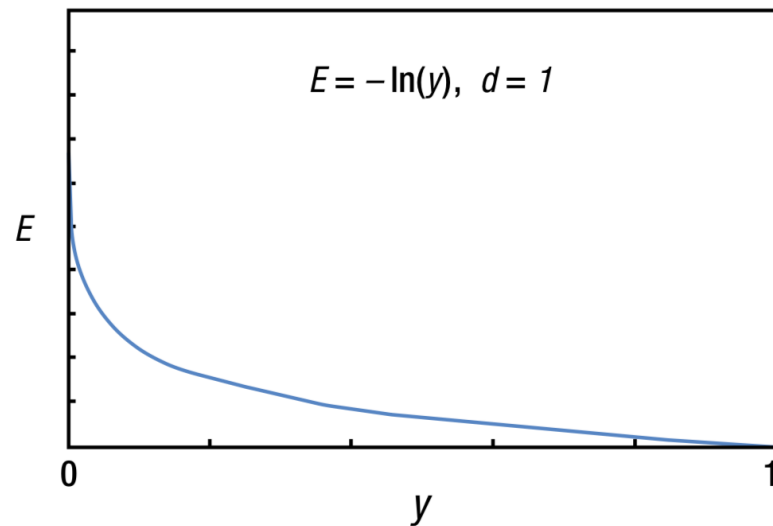
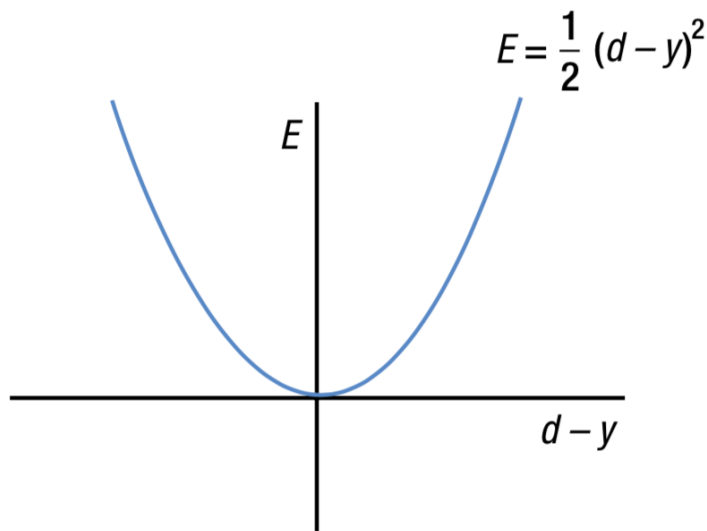
- 代价函数和学习规则

平方和

$$J = \sum_{i=1}^M \frac{1}{2} (d_i - y_i)^2$$

交叉熵

$$J = \sum_{i=1}^M \{-d_i \ln(y_i) - (1-d_i) \ln(1-y_i)\}$$



训练多层神经网络

$$L = -d\ln(y) - (1 - d)\ln(1 - y)$$

$$\Rightarrow \frac{\partial L}{\partial y} = -d \cdot \frac{1}{y} + (1 - d) \frac{1}{1 - y}$$

$$\begin{aligned}\Rightarrow \delta &= \frac{\partial L}{\partial y} \frac{\partial y}{\partial v} \\&= \left(-d \cdot \frac{1}{y} + (1 - d) \frac{1}{1 - y} \right) \varphi(v)(1 - \varphi(v)) \\&= \left(-d \cdot \frac{1}{y} + (1 - d) \frac{1}{1 - y} \right) y(1 - y) \\&= (-d(1 - y) + (1 - d)y) \\&= -(d - y)\end{aligned}$$

$$\begin{aligned}\mathbf{w}_{n+1} &= \mathbf{w}_n - \alpha \frac{\partial L}{\partial \mathbf{w}_n} \\ &= \mathbf{w}_n - \alpha \frac{\partial L}{\partial y} \frac{\partial y}{\partial v} \frac{\partial v}{\partial \mathbf{w}_n} \\ &= \mathbf{w}_n - \alpha(-1)(d - y)\mathbf{x} \\ &= \mathbf{w}_n + \alpha e \mathbf{x}\end{aligned}$$

训练多层神经网络

- 用Sigmoid激活和交叉熵代价的后向传播算法

1. 初始化权重

2. 将输入值送入神经网络，计算输出值，和正确输出值的误差，以及输出节点的Delta

$$e = d - y$$

$$\delta = e$$

3. 将输出节点的Delta后向传播，用于计算相邻节点的Delta

$$e^{(k)} = W^T \delta$$

$$\delta^{(k)} = \varphi'(v^{(k)})e^{(k)}$$

4. 重复步骤3直至到达输入节点之后的第一个隐含层

5. 根据以下规则更新权重

$$\Delta w_{ij} = \alpha \delta_i x_j$$

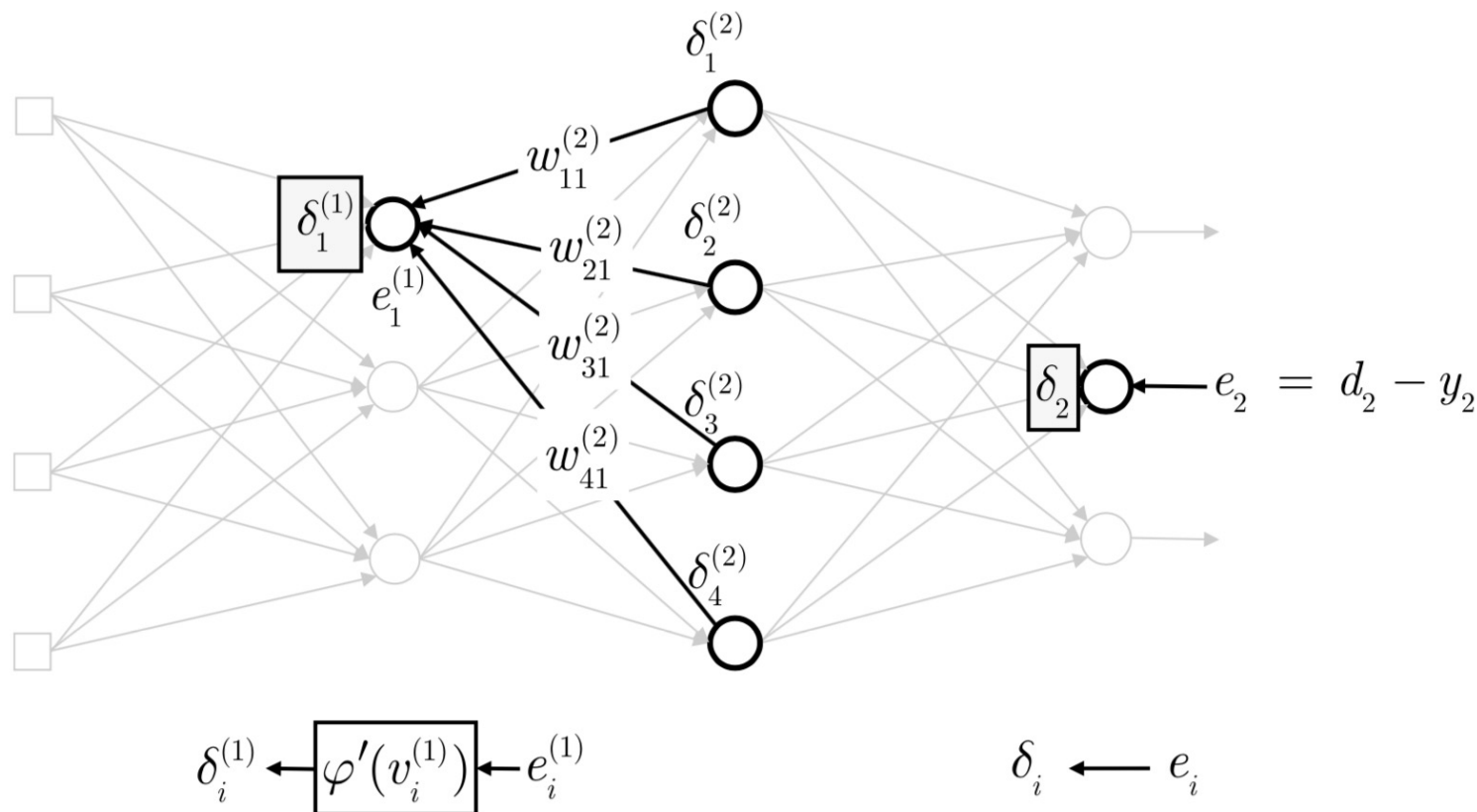
$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$

6. 对所有训练数据执行步骤2-5

7. 重复步骤2-6直到误差小于给定阈值

训练多层神经网络

- 用Sigmoid激活和交叉熵代价的后向传播算法



训练多层神经网络

- 过拟合和正则化

正则化的
平方和代价

$$J = \frac{1}{2} \sum_{i=1}^M (d_i - y_i)^2 + \lambda \frac{1}{2} \|w\|^2$$

正则化的
交叉熵代价

$$J = \sum_{i=1}^M \{-d_i \ln(y_i) - (1-d_i) \ln(1-y_i)\} + \lambda \frac{1}{2} \|w\|^2$$

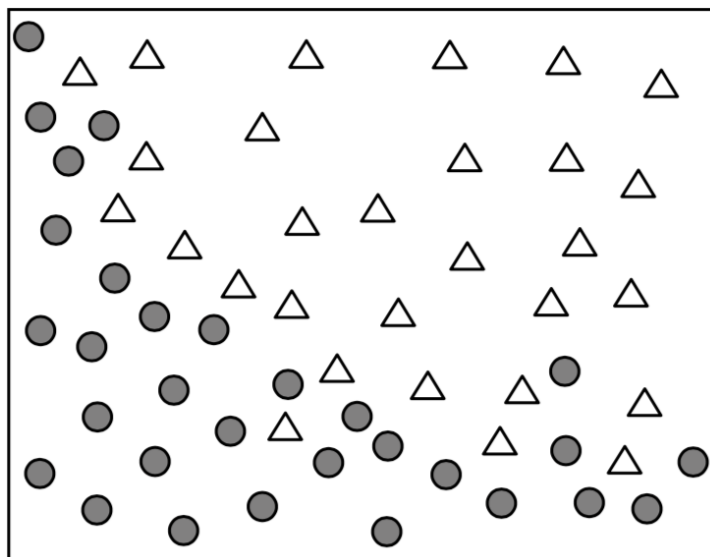
主要内容

- 机器学习
- 神经网络
- 训练多层神经网络
- 用于分类的多层神经网络
- 深度学习
- 卷积神经网络

用于分类的多层神经网络

- 二分类

训练数据



$\{5, 7, \triangle\}$
$\{9, 8, \bullet\}$

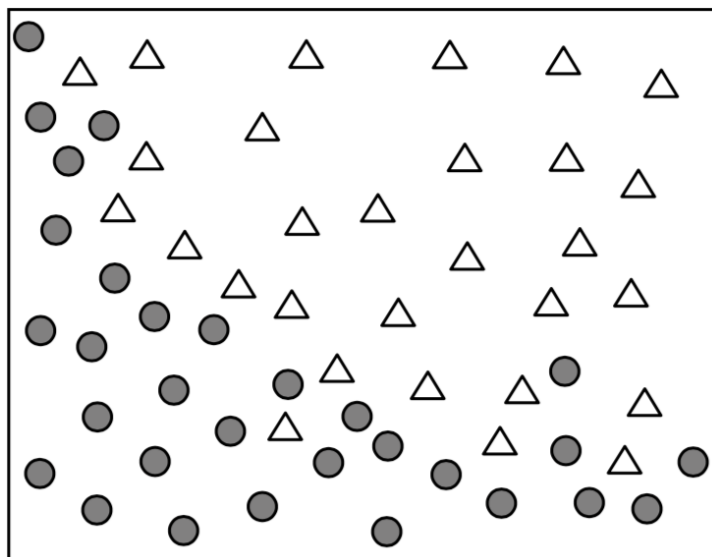
...

$\{6, 5, \bullet\}$

用于分类的多层神经网络

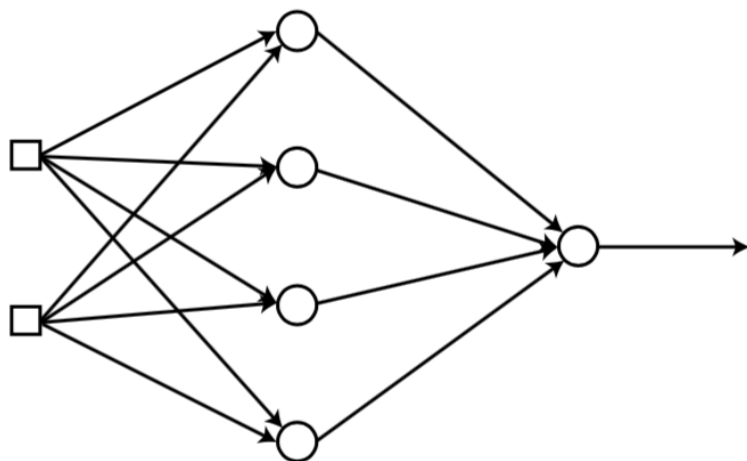
- 二分类

训练数据



{5, 7, Δ }
{9, 8, $\bullet$ }
...
{6, 5, $\bullet$ }

单输出
网络



Class $\Delta \rightarrow 1$

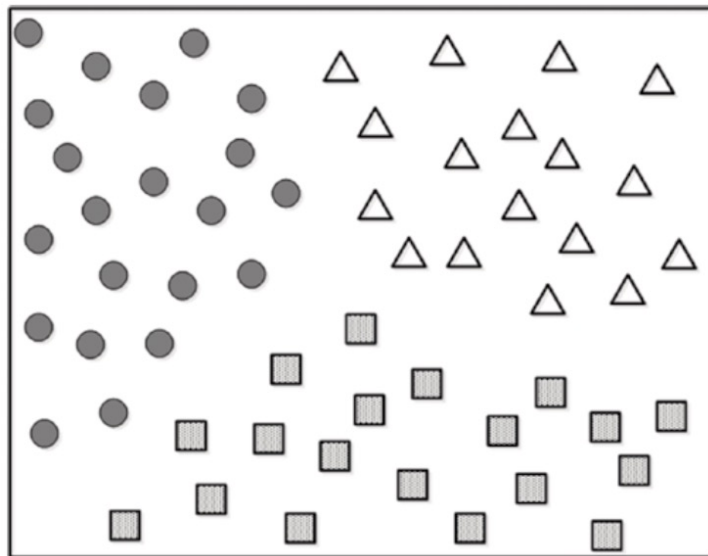
Class $\bullet \rightarrow 0$

定义
网络
输出

用于分类的多层神经网络

- 多分类

训练数据

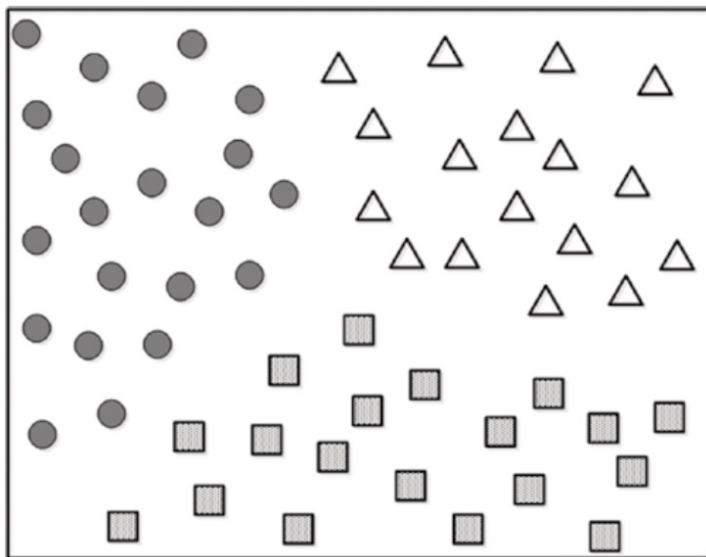


{ 5, 7, Class1 }
{ 9, 8, Class 3 }
{ 2, 4, Class 2 }
...
{ 6, 5, Class 3 }

用于分类的多层神经网络

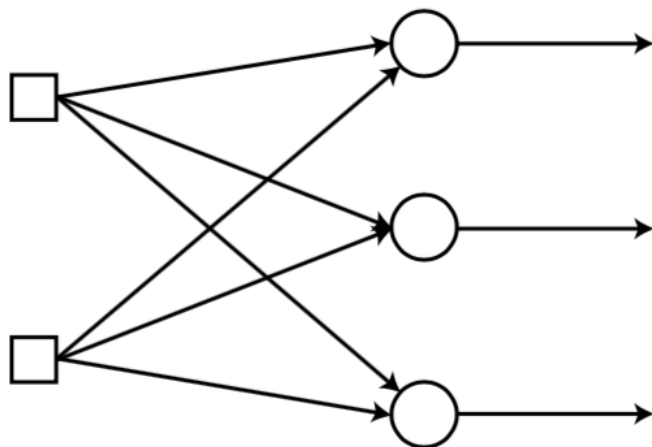
- 多分类

训练数据



{ 5, 7, Class1 }
{ 9, 8, Class 3 }
{ 2, 4, Class 2 }
...
{ 6, 5, Class 3 }

多输出
网络



Class 1 $\rightarrow [1\ 0\ 0]$

Class 2 $\rightarrow [0\ 1\ 0]$

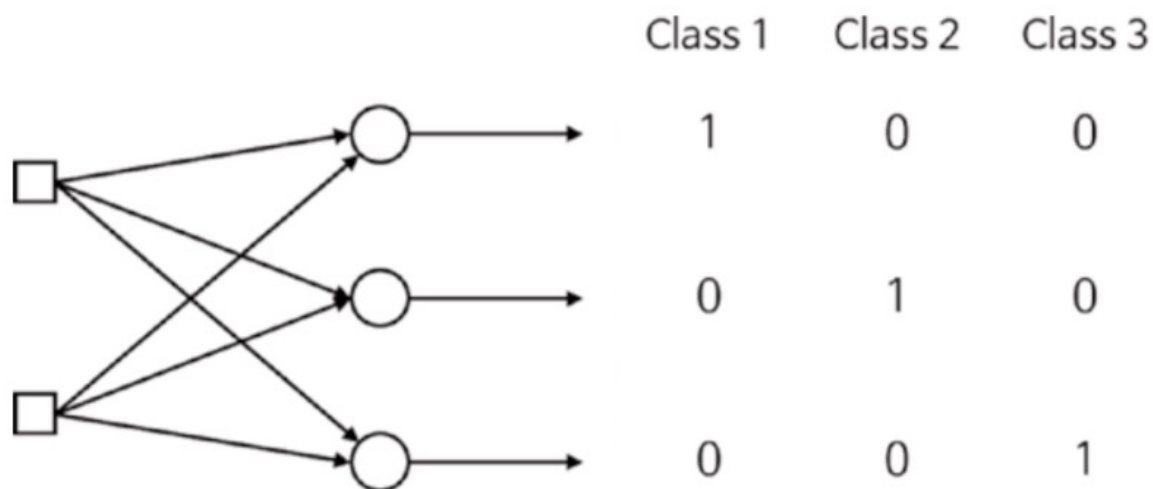
Class 3 $\rightarrow [0\ 0\ 1]$

定义
网络
输出

用于分类的多层神经网络

- 多分类

定义网络输出



训练数据

{ 5, 7, 1, 0, 0 }
{ 9, 8, 0, 0, 1 }

{ 2, 4, 0, 1, 0 }

...

{ 6, 5, 0, 0, 1 }

用于分类的多层神经网络

- 多分类
 - 激活函数选用Softmax
 - 某个节点输出不仅由输入决定，还和其他节点的输出有关

$$v = \begin{bmatrix} 2 \\ 1 \\ 0.1 \end{bmatrix} \Rightarrow \varphi(v) = \begin{bmatrix} \frac{e^2}{e^2 + e^1 + e^{0.1}} \\ \frac{e^1}{e^2 + e^1 + e^{0.1}} \\ \frac{e^{0.1}}{e^2 + e^1 + e^{0.1}} \end{bmatrix} = \begin{bmatrix} 0.6590 \\ 0.2424 \\ 0.0986 \end{bmatrix}$$

用于分类的多层神经网络

- 多分类
 - 激活函数选用Softmax
 - 某个节点输出不仅由输入决定，还和其他节点的输出有关

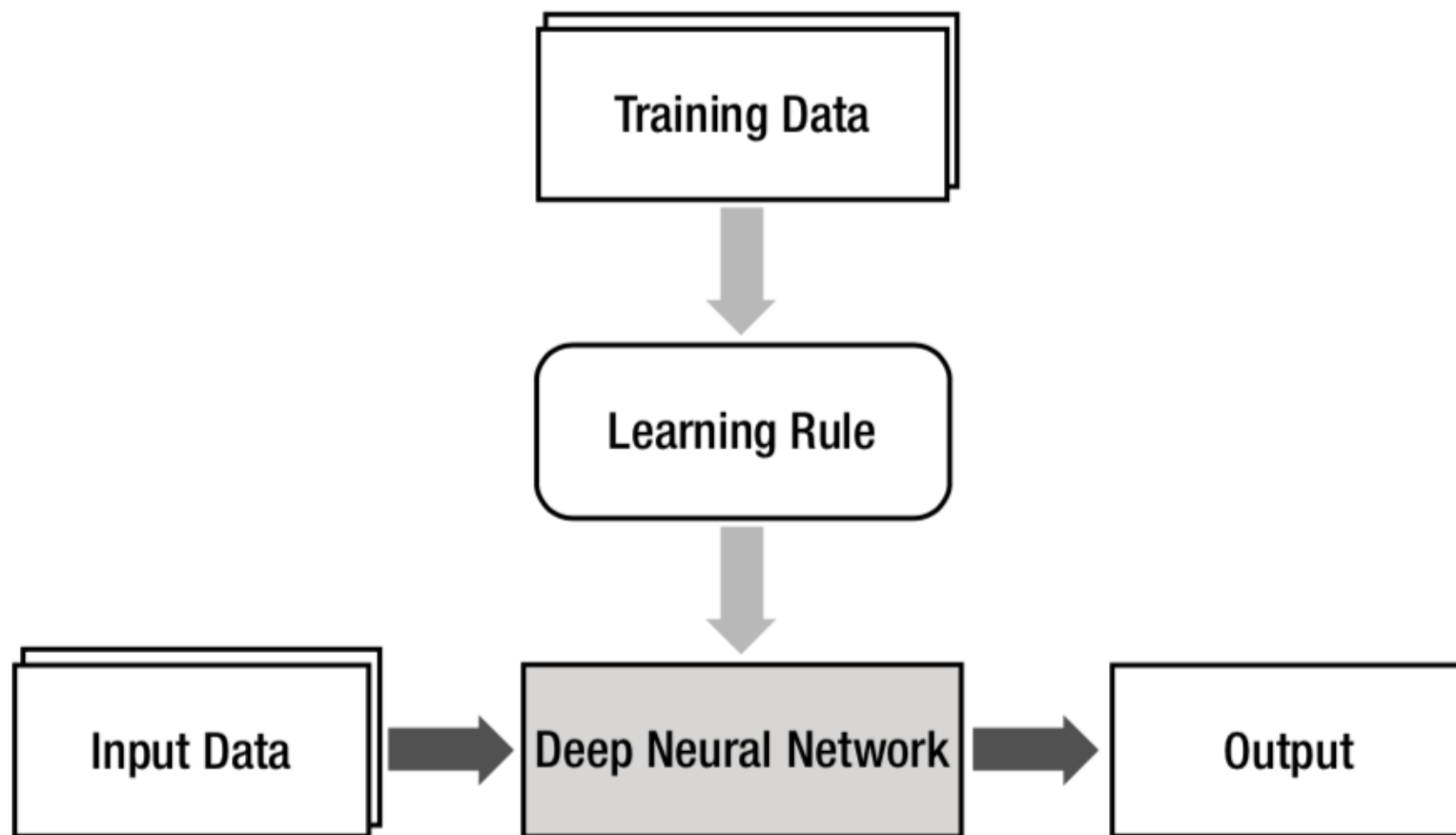
$$v = \begin{bmatrix} 2 \\ 1 \\ 0.1 \end{bmatrix} \Rightarrow \varphi(v) = \begin{bmatrix} \frac{e^2}{e^2 + e^1 + e^{0.1}} \\ \frac{e^1}{e^2 + e^1 + e^{0.1}} \\ \frac{e^{0.1}}{e^2 + e^1 + e^{0.1}} \end{bmatrix} = \begin{bmatrix} 0.6590 \\ 0.2424 \\ 0.0986 \end{bmatrix}$$

$$y_i = \varphi(v_i) = \frac{e^{v_i}}{e^{v_1} + e^{v_2} + e^{v_3} + \dots + e^{v_M}}$$
$$= \frac{e^{v_i}}{\sum_{k=1}^M e^{v_k}}$$

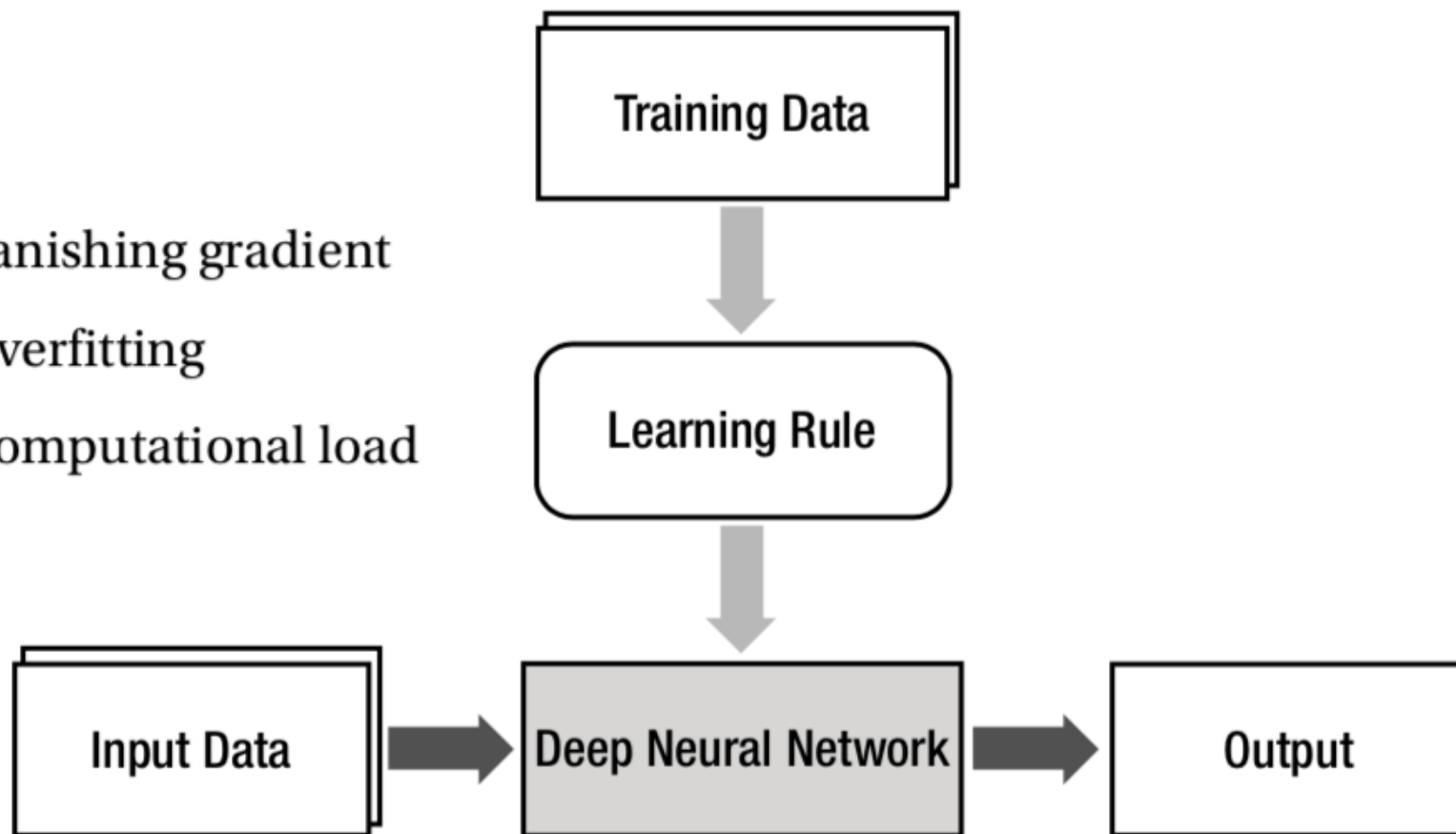
$$\varphi(v_1) + \varphi(v_2) + \varphi(v_3) + \dots + \varphi(v_M) = 1$$

主要内容

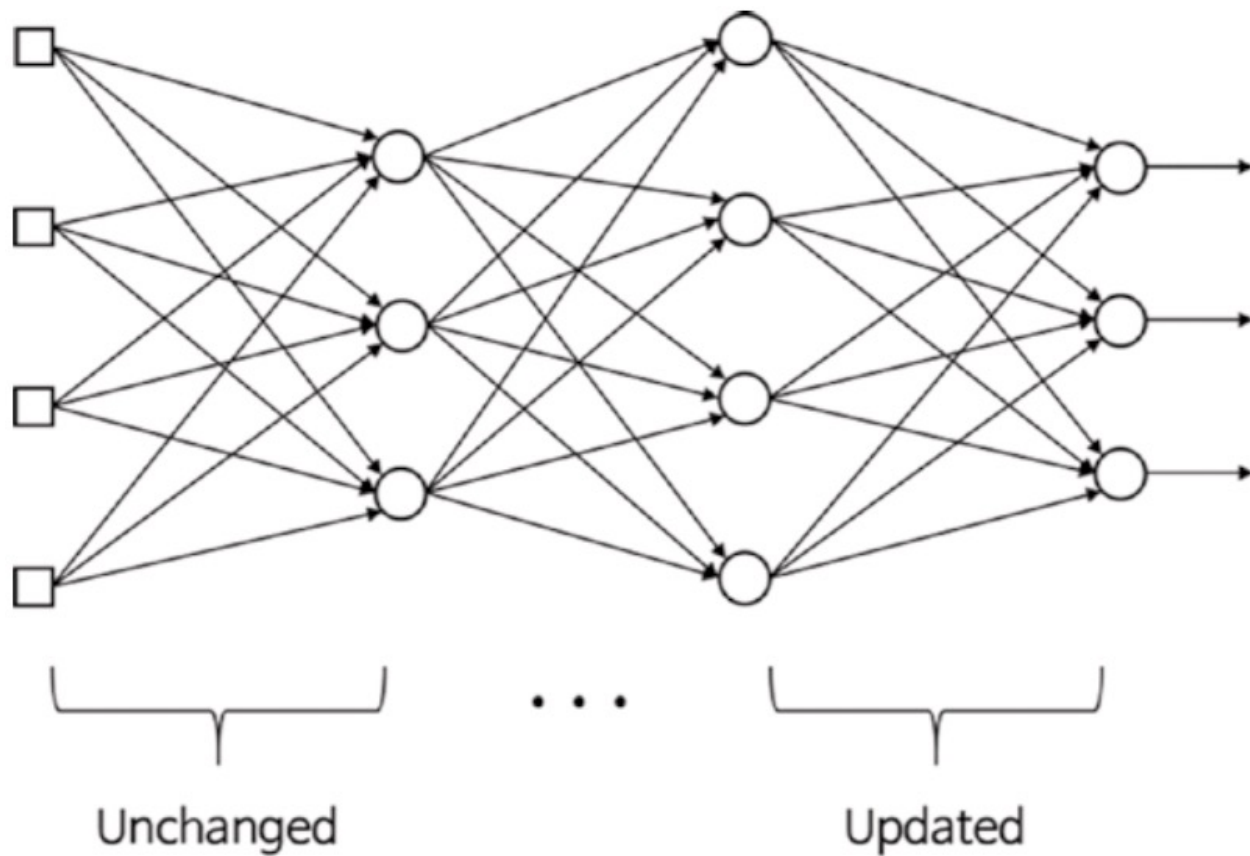
- 机器学习
- 神经网络
- 训练多层神经网络
- 用于分类的多层神经网络
- 深度学习
- 卷积神经网络



- Vanishing gradient
- Overfitting
- Computational load

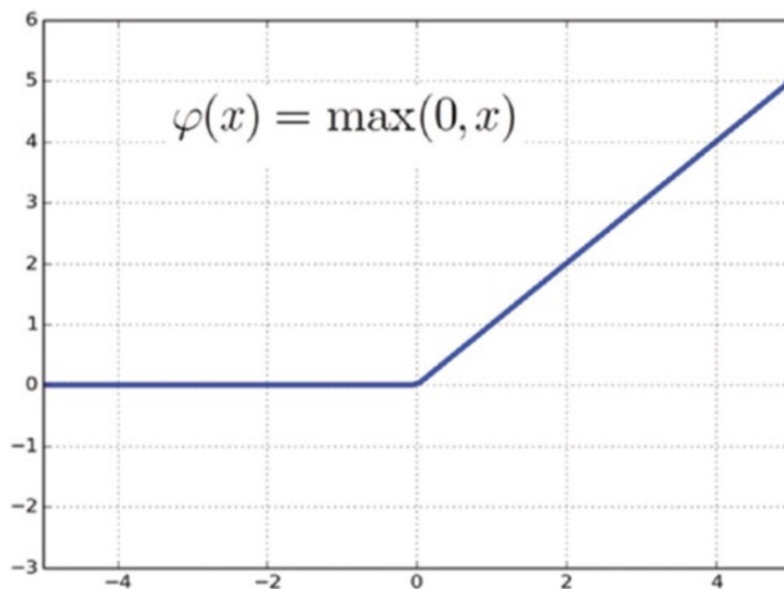


- 梯度消失

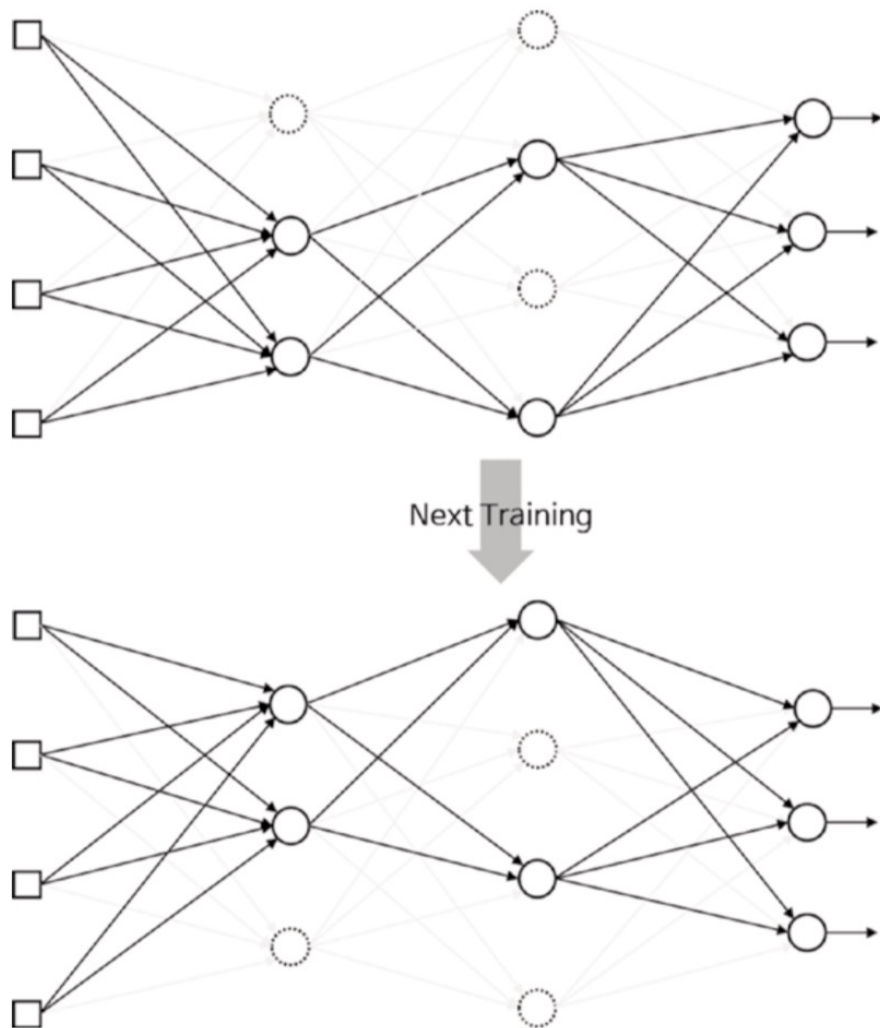


- 梯度消失
 - 解决方法：ReLU激活函数

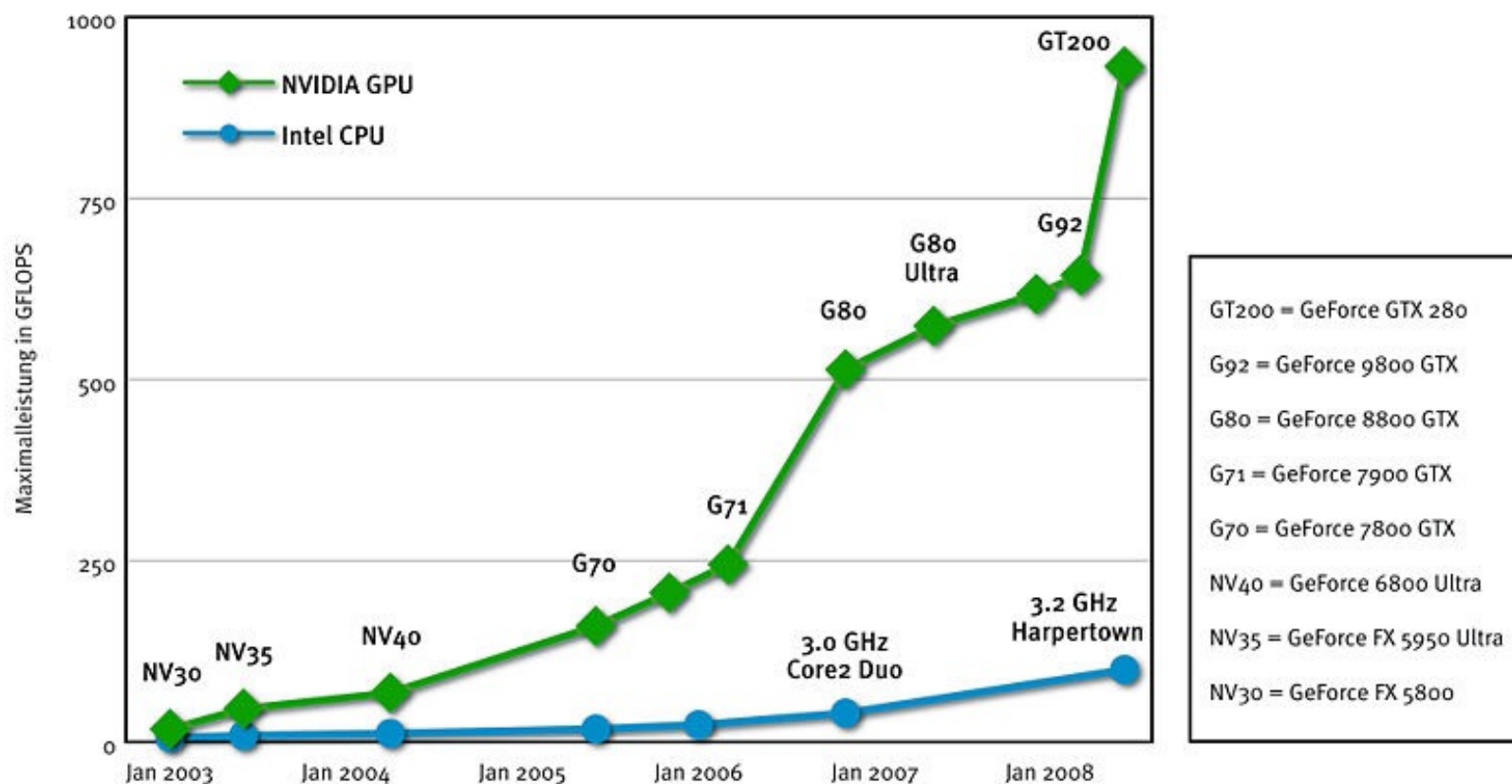
$$\begin{aligned}\varphi(x) &= \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases} \\ &= \max(0, x)\end{aligned}$$



- 过拟合
 - 解决方法: dropout



- 计算量大
 - 解决方法: GPU



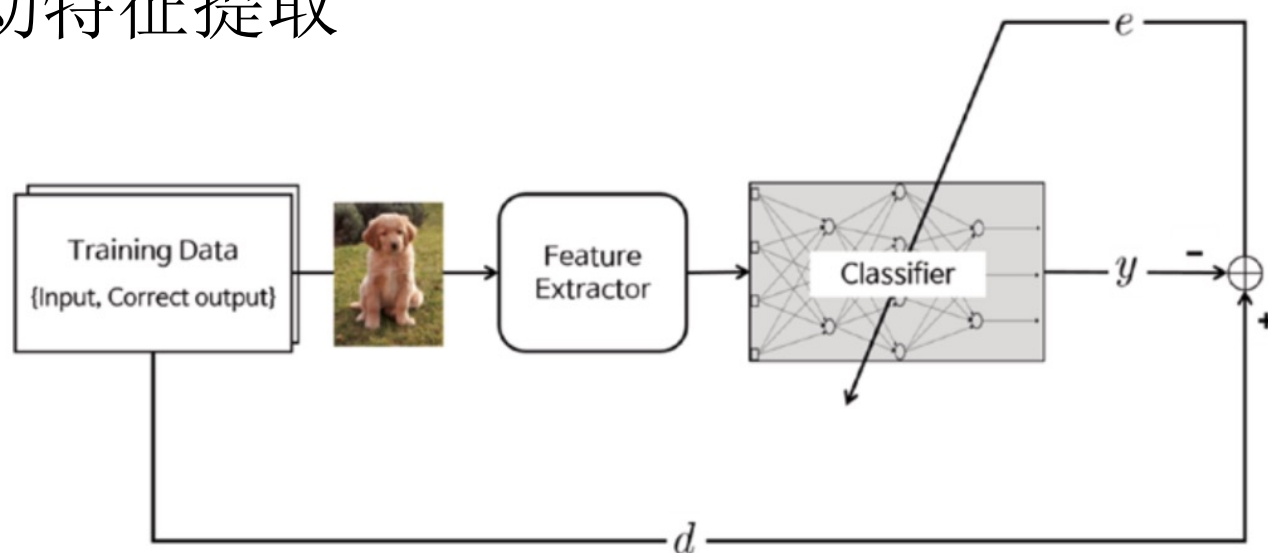
主要内容

- 机器学习
- 神经网络
- 训练多层神经网络
- 用于分类的多层神经网络
- 深度学习
- 卷积神经网络

卷积神经网络

- 特点：自动特征提取

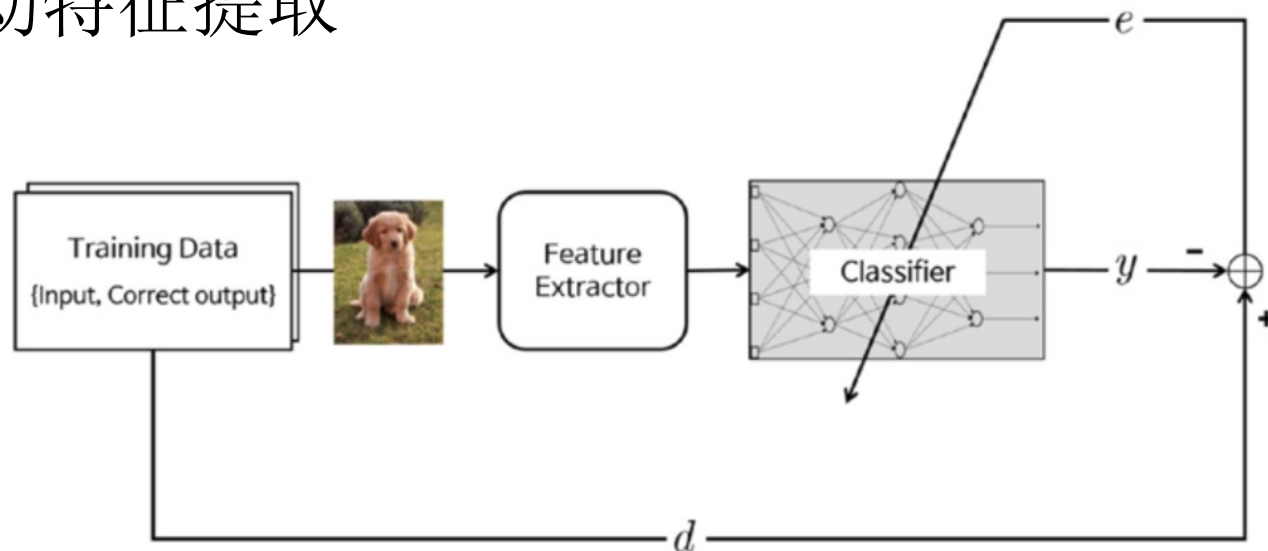
传统机器学习
分类器



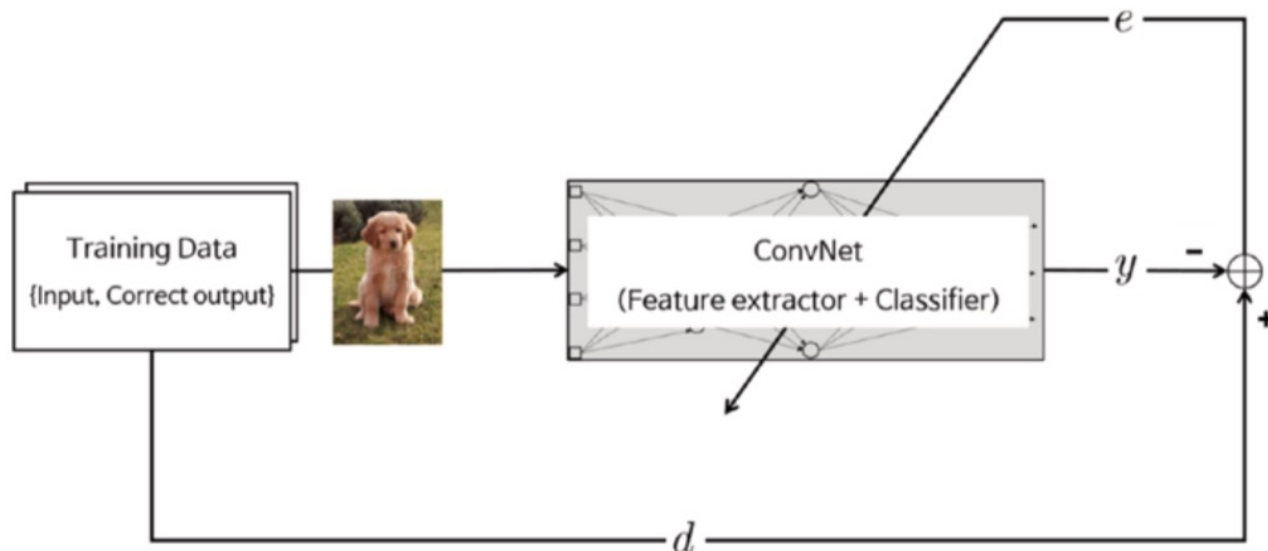
卷积神经网络

- 特点：自动特征提取

传统机器学习
分类器

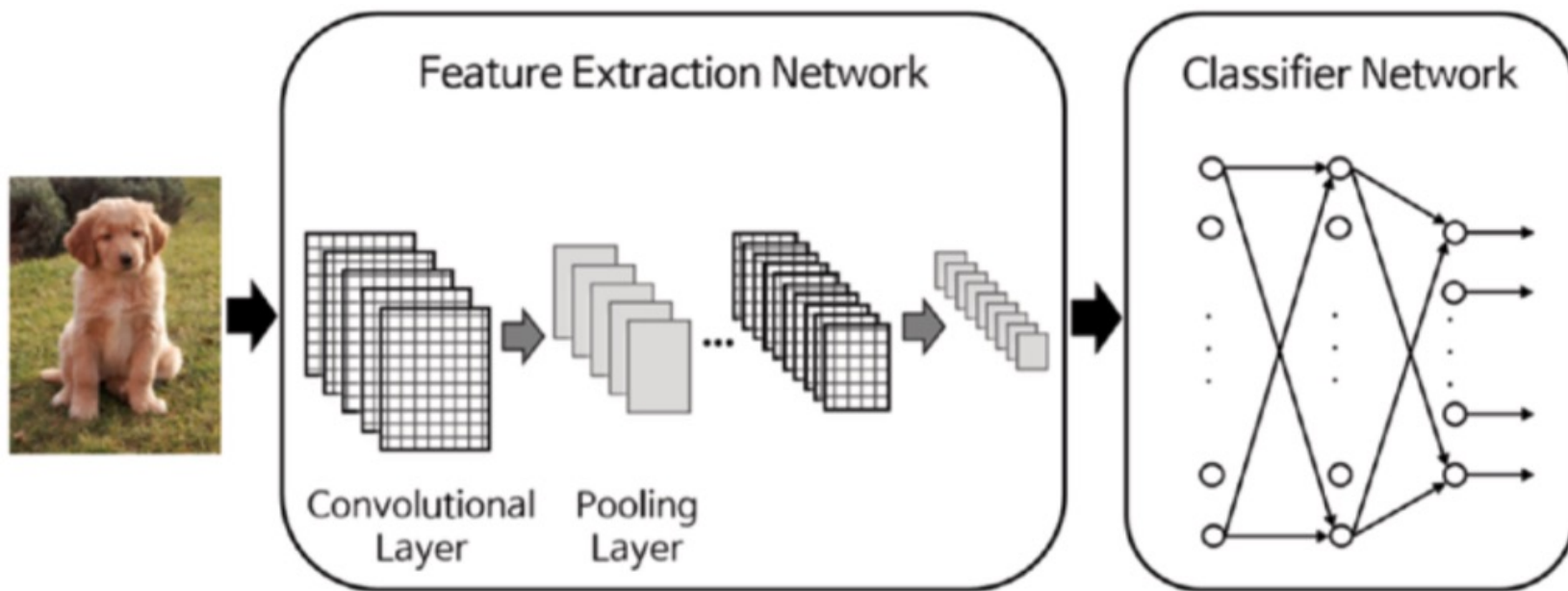


基于卷积神经
网络的分类器



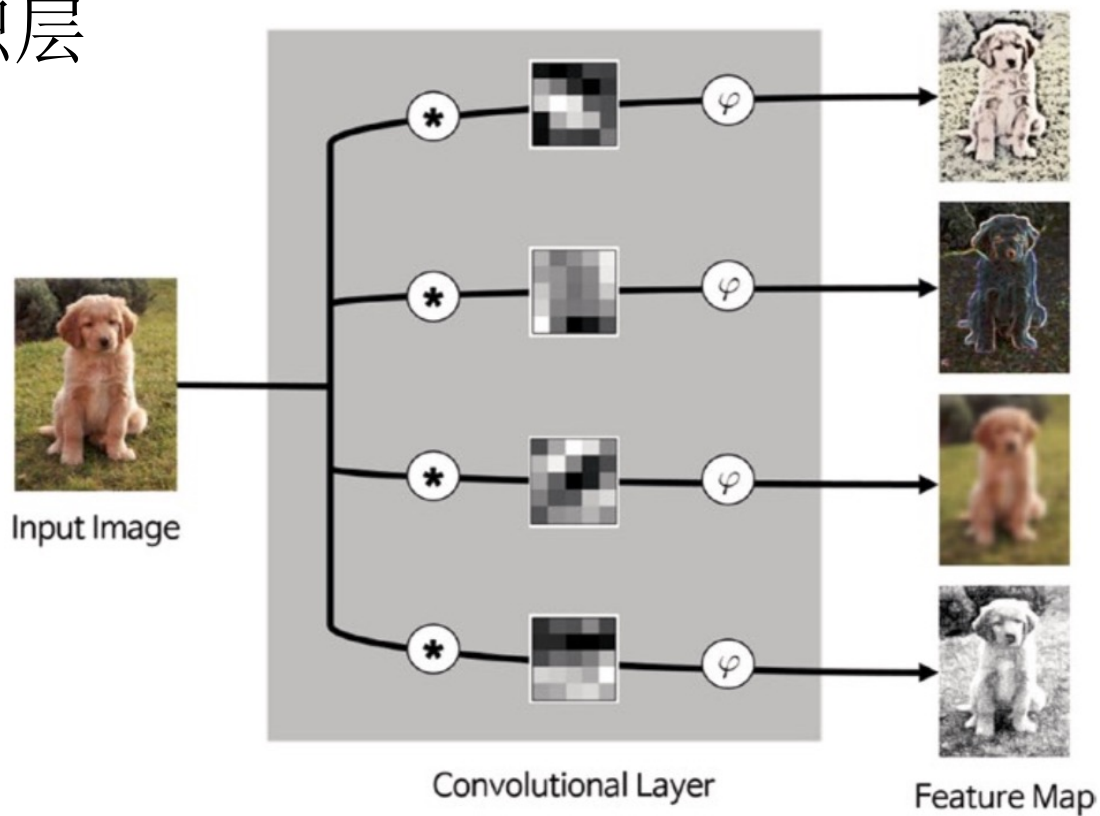
卷积神经网络

- 典型结构



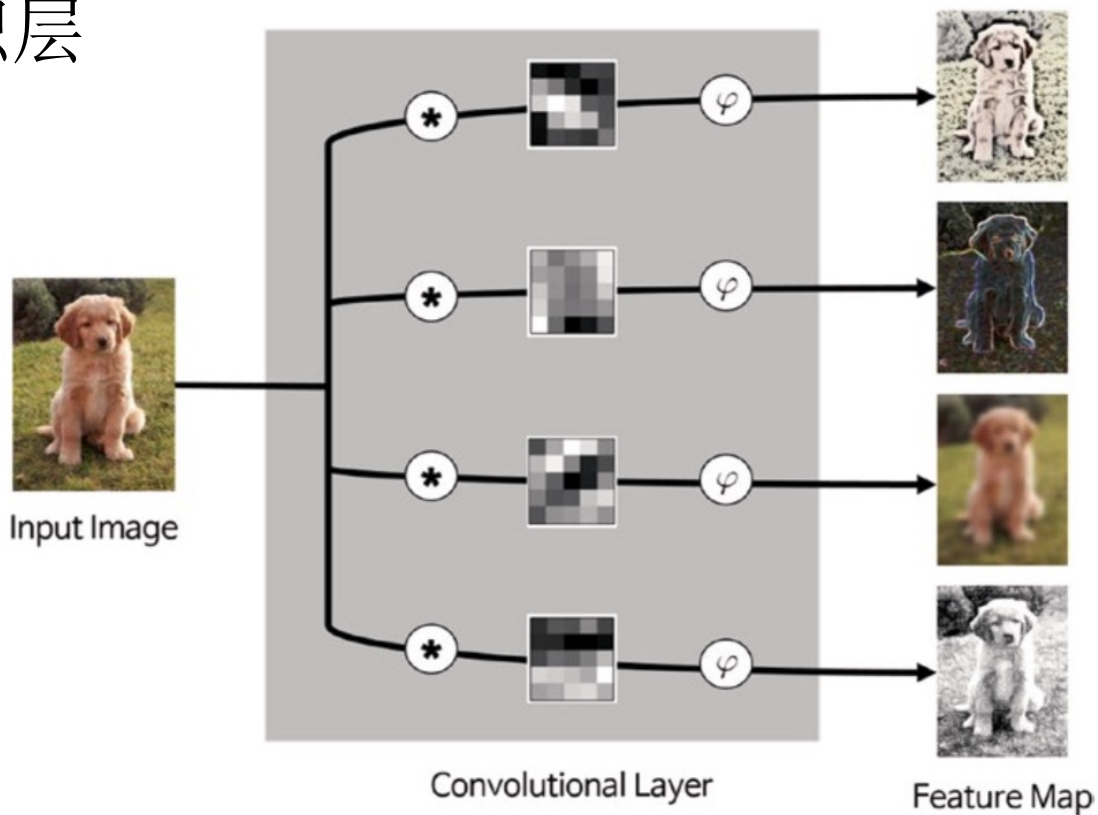
卷积神经网络

- 卷积层



卷积神经网络

- 卷积层



1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

卷积神经网络

- 卷积层

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4



$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

=

7		

$$(1 \times 1) + (1 \times 0) + (4 \times 0) + (6 \times 1) = 7$$

卷积神经网络

- 卷积层

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast$ $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ $=$

7		

$$(1 \times 1) + (1 \times 0) + (4 \times 0) + (6 \times 1) = 7$$

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast$ $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ $=$

7	5	

卷积神经网络

- 卷积层

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7		

$$(1 \times 1) + (1 \times 0) + (4 \times 0) + (6 \times 1) = 7$$

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7	5	

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7	5	9

卷积神经网络

- 卷积层

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7		

$$(1 \times 1) + (1 \times 0) + (4 \times 0) + (6 \times 1) = 7$$

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7	5	

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7	5	9

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7	5	9
4		

卷积神经网络

- 卷积层

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7		

$$(1 \times 1) + (1 \times 0) + (4 \times 0) + (6 \times 1) = 7$$

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7	5	

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7	5	9

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7	5	9
4		

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =$

7	5	9
4	7	9
32	2	5

卷积神经网络

- 卷积层

图像和滤波器匹配时输出值大

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast$

1	0
0	1

 $=$

7	5	9
4	7	9
32	2	5

卷积神经网络

- 卷积层

图像和滤波器匹配时输出值大

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\odot$

1	0
0	1

 $=$

7	5	9
4	7	9
32	2	5

不匹配时输出值小

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\odot$

1	0
0	1

 $=$

7	5	9
4	7	9
32	2	5

卷积神经网络

- 卷积层

图像和滤波器匹配时输出值大

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast$
$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$
 $=$

7	5	9
4	7	9
32	2	5

不匹配时输出值小

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast$
$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$
 $=$

7	5	9
4	7	9
32	2	5

换个滤波器考察

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

 $\circledast$
$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$
 $=$

5	7	7
36	4	9
0	3	7

卷积神经网络

• 卷积层的后向传播

X_{11}	X_{12}	X_{13}
X_{21}	X_{22}	X_{23}
X_{31}	X_{32}	X_{33}

W_{11}	W_{12}
W_{21}	W_{22}

X_{11}	X_{12}	X_{13}
X_{21}	X_{22}	X_{23}
X_{31}	X_{32}	X_{33}

W_{11}	W_{12}
W_{21}	W_{22}

X_{11}	X_{12}	X_{13}
X_{21}	X_{22}	X_{23}
X_{31}	X_{32}	X_{33}

W_{11}	W_{12}
W_{21}	W_{22}

$$W_{11}X_{11} + W_{12}X_{12} + W_{21}X_{21} + W_{22}X_{22}$$

$$W_{11}X_{12} + W_{12}X_{13} + W_{21}X_{22} + W_{22}X_{23}$$

$$W_{11}X_{21} + W_{12}X_{22} + W_{21}X_{31} + W_{22}X_{32}$$

$$W_{11}X_{22} + W_{12}X_{23} + W_{21}X_{32} + W_{22}X_{33}$$

$$W_{11}X_{11} + W_{12}X_{12} + W_{21}X_{21} + W_{22}X_{22}$$

--

--

$$W_{11}X_{11} + W_{12}X_{12} + W_{21}X_{21} + W_{22}X_{22}$$

$$W_{11}X_{12} + W_{12}X_{13} + W_{21}X_{22} + W_{22}X_{23}$$

卷积神经网络

- 卷积层的后向传播

卷积表达式

$$h_{11} = W_{11}X_{11} + W_{12}X_{12} + W_{21}X_{21} + W_{22}X_{22}$$

$$h_{12} = W_{11}X_{12} + W_{12}X_{13} + W_{21}X_{22} + W_{22}X_{23}$$

$$h_{21} = W_{11}X_{21} + W_{12}X_{22} + W_{21}X_{31} + W_{22}X_{32}$$

$$h_{22} = W_{11}X_{22} + W_{12}X_{23} + W_{21}X_{32} + W_{22}X_{33}$$

梯度记法

$$\partial h_{ij} \text{ represents } \frac{\partial L}{\partial h_{ij}}$$

$$\partial w_{ij} \text{ represents } \frac{\partial L}{\partial w_{ij}}$$

卷积神经网络

• 卷积层的后向传播

x_{11}	x_{12}	x_{13}
x_{21}	x_{22}	x_{23}
x_{31}	x_{32}	x_{33}

∂h_{11}
∂h_{21}

$x_{11} \partial h_{11} +$ $x_{12} \partial h_{12} +$ $x_{21} \partial h_{21} +$ $x_{22} \partial h_{22}$	$x_{12} \partial h_{11} +$ $x_{13} \partial h_{12} +$ $x_{22} \partial h_{21} +$ $x_{23} \partial h_{22}$
--	--

x_{11}	x_{12}	x_{13}
x_{21}	x_{22}	x_{23}
x_{31}	x_{32}	x_{33}

$x_{11} \partial h_{11}$	$x_{12} \partial h_{11}$
$x_{21} \partial h_{11}$	$x_{22} \partial h_{11}$

∂h_{11}	
∂h_{21}	∂h_{22}

$x_{21} \partial h_{11} +$ $x_{22} \partial h_{12} +$ $x_{31} \partial h_{21} +$ $x_{32} \partial h_{22}$	$x_{22} \partial h_{11} +$ $x_{23} \partial h_{12} +$ $x_{32} \partial h_{21} +$ $x_{33} \partial h_{22}$
--	--

x_{11}	x_{12}	x_{13}
x_{21}	x_{22}	x_{23}
x_{31}	x_{32}	x_{33}

$x_{11} \partial h_{11} +$ $x_{12} \partial h_{12} +$ $x_{21} \partial h_{21} +$ $x_{22} \partial h_{22}$	$x_{12} \partial h_{11} +$ $x_{13} \partial h_{12} +$ $x_{22} \partial h_{21} +$ $x_{23} \partial h_{22}$
$x_{21} \partial h_{11} +$ $x_{22} \partial h_{12} +$ $x_{31} \partial h_{21} +$ $x_{32} \partial h_{22}$	$x_{22} \partial h_{11} +$ $x_{23} \partial h_{12} +$ $x_{32} \partial h_{21} +$ $x_{33} \partial h_{22}$

∂h_{11}	∂h_{12}
∂h_{21}	∂h_{22}

卷积神经网络

- 卷积层的后向传播

$$\partial W_{11} = X_{11} \partial h_{11} + X_{12} \partial h_{12} + X_{21} \partial h_{21} + X_{22} \partial h_{22}$$

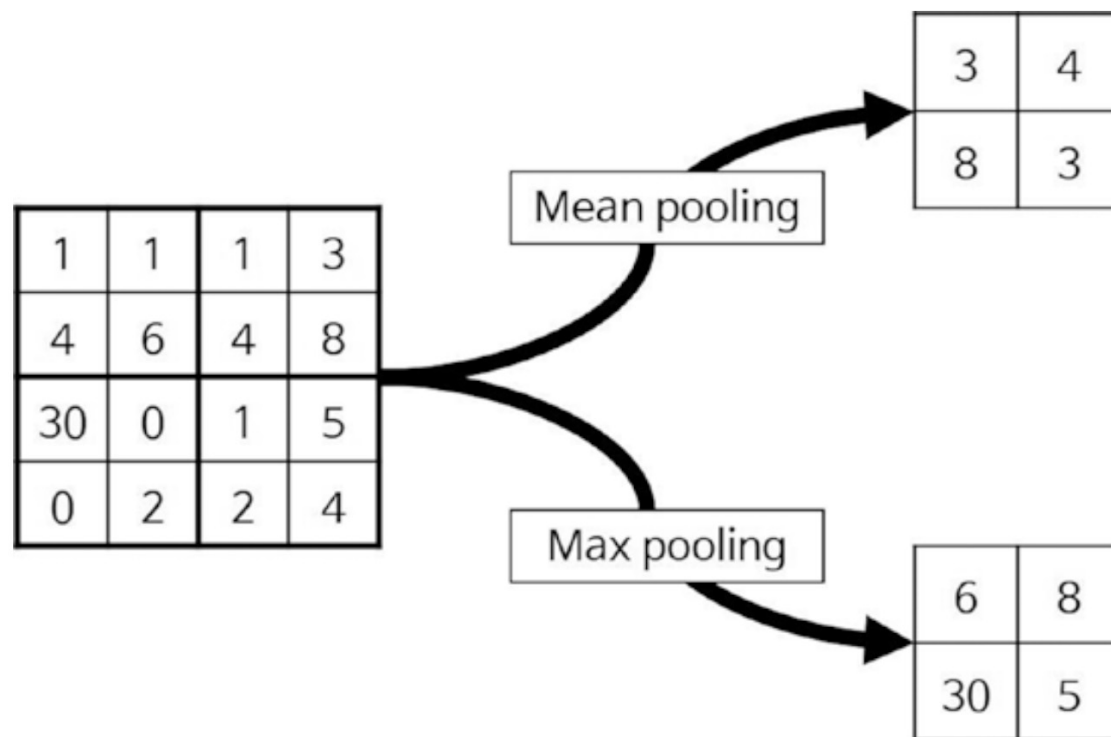
$$\partial W_{12} = X_{12} \partial h_{11} + X_{13} \partial h_{12} + X_{22} \partial h_{21} + X_{23} \partial h_{22}$$

$$\partial W_{21} = X_{21} \partial h_{11} + X_{22} \partial h_{12} + X_{31} \partial h_{21} + X_{32} \partial h_{22}$$

$$\partial W_{22} = X_{22} \partial h_{11} + X_{23} \partial h_{12} + X_{32} \partial h_{21} + X_{33} \partial h_{22}$$

卷积神经网络

- 池化层



- 池化层的后向传播

Max-Pooling Layer

Forward :

```
for (p = 0; p < k; p++)
```

```
  for (q = 0; q < k; q++)
```

```
     $y_n(x, y) = \max(y_n(x, y), y_{n-1}(x + p, y + q));$ 
```



Backward:

$$\frac{\partial L}{\partial y_{n-1}}(x + p, y + q) = \begin{cases} = 0, & \text{if } (y_n(x, y) \neq y_{n-1}(x + p, y + q)) \\ = \frac{\partial L}{\partial y_n}(x, y), & \text{otherwise} \end{cases}$$

卷积神经网络

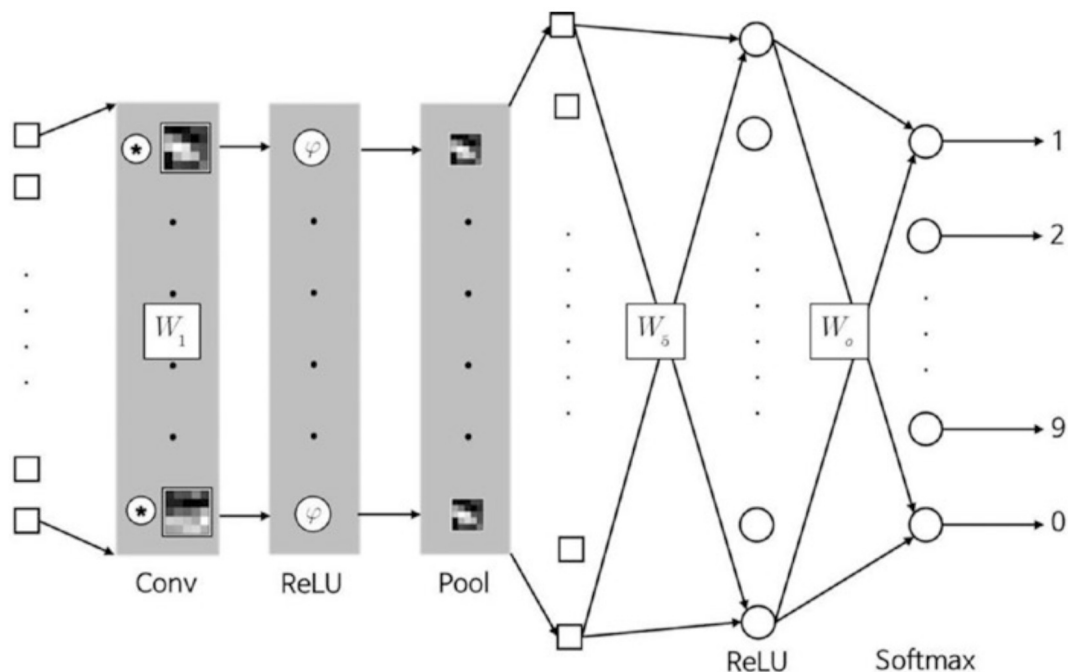
- 举例：MNIST



卷积神经网络

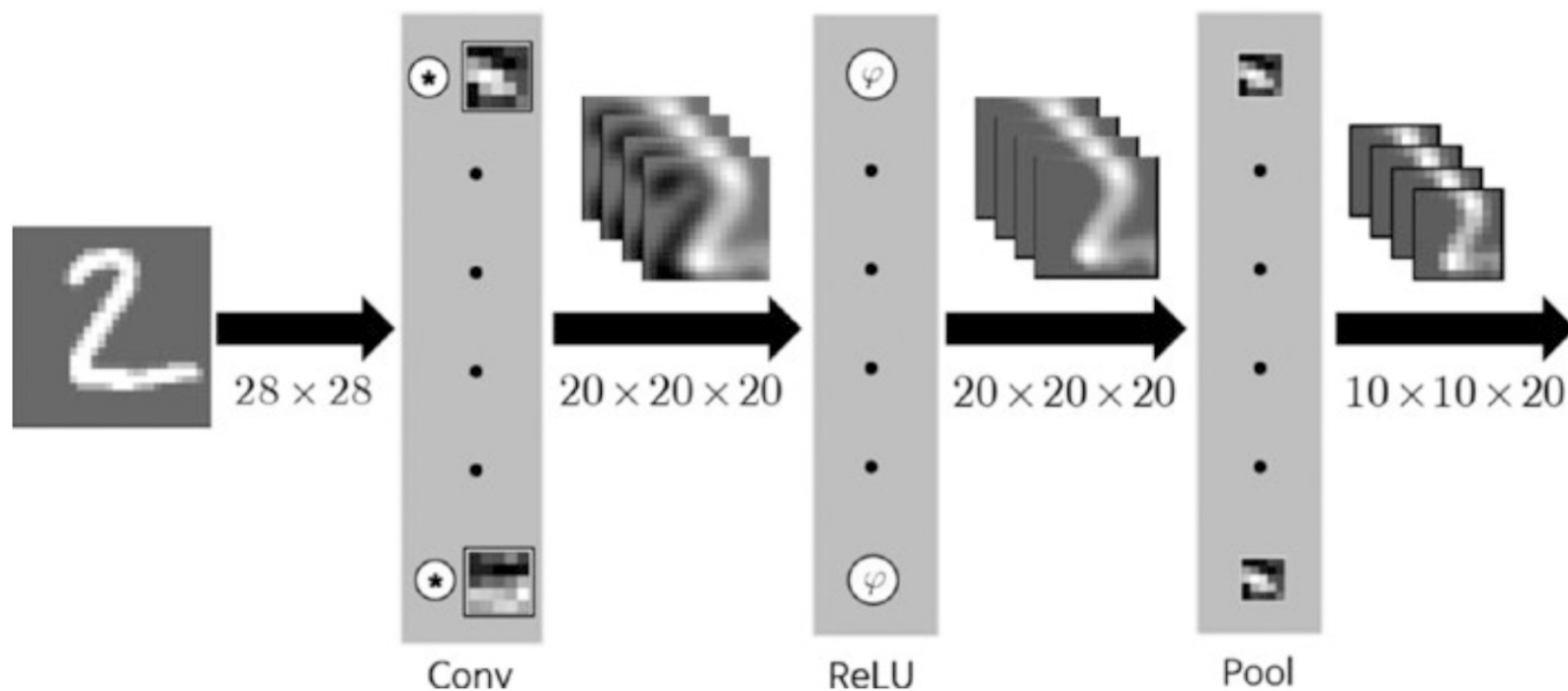
- 举例：MNIST

Layer	Remark	Activation Function
Input	28×28 nodes	-
Convolution	20 convolution filters (9×9)	ReLU
Pooling	1 mean pooling (2×2)	-
Hidden	100 nodes	ReLU
Output	10 nodes	Softmax



卷积神经网络

- 举例：MNIST

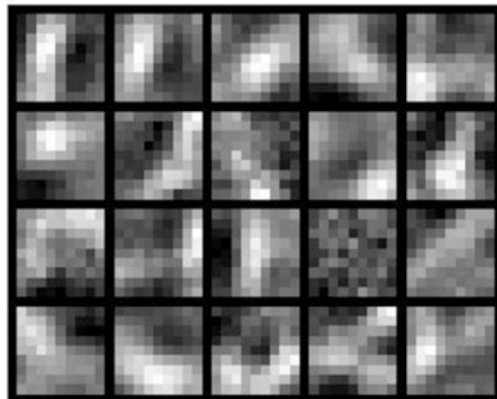


卷积神经网络

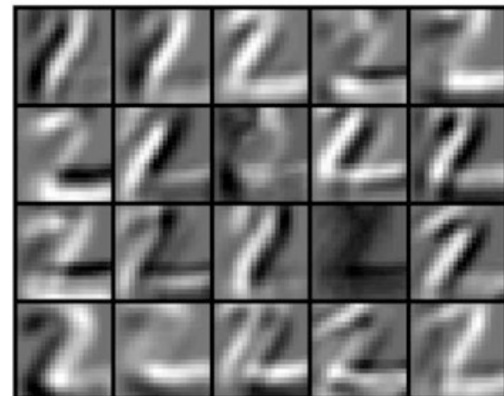
- 举例：MNIST



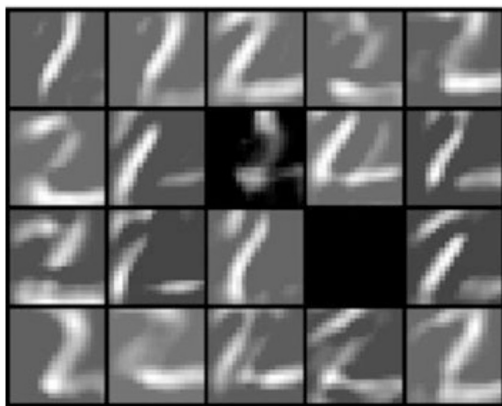
输入图像



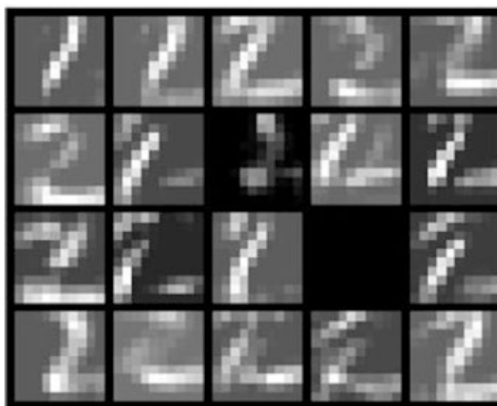
20个训好的
滤波器



滤波器的
输出



ReLU的输出



均值池化的
输出

感谢同学们认真听课！

